

Roof Engine Project Specification

Project – Roof

Data Foundation Lab
DFLab FHO Pty Ltd – Proprietary Limited Company
(13 676 090 289)

Table of Contents

1. Overview *6*

1.1 Purpose 6

1.2 Scope 7

1.3 Role within the Roof Ecosystem 7

2. Functional Responsibilities *8*

2.1 Core Responsibilities 8

2.1.1 System Lifecycle 8

2.1.2 Client Management 8

2.1.3 File-System Management 9

2.1.4 Networking 9

2.1.5 Security 10

2.1.6 Updates & Maintenance 10

2.2 Out-of-Scope Responsibilities 10

3. System Architecture *10*

3.1 High-Level Architecture 10

3.2 Process Startup 11

3.3 Runtime Service Model 12

3.4 Main Modules 12

3.5 Roof.Common Dependency 13

4. Runtime Lifecycle *14*

4.1 Boot Sequence 14

4.2 System Checks 15

4.3 Service Initialisation 15

4.4 Main Loop 16

4.5 Shutdown Behaviour 16

5. Client State Model *17*

5.1 Runtime Client Object 17

5.2 Client Collections 17

5.3 Client Verification State 17

5.4 Encryption Info Lookup 18

6. Configuration Model *18*

6.1 Configuration File 18

6.2 Configuration Schema 18

6.3 Registered Client Configuration 19

6.4 Config Save Behaviour 19

6.5 Config Load Behaviour 19

6.6 Config Migration **20**
6.7 Config Security Requirements **21**
7. Data Model & Storage ***22***
7.1 Shared Storage Root **22**
7.2 File-System Metadata Model **22**
7.3 Directory Metadata **22**
7.4 File Metadata **23**
7.5 Indexing Strategy **23**
7.6 Data Consistency Boundaries **24**
8. File-System Behaviour ***24***
8.1 Initial Index Construction **24**
8.2 Directory Operations **24**
8.2.1 Get Directory 24
8.2.2 Create Directory **25**
8.2.3 Update Directory 25
8.2.4 Copy Directory 25
8.2.5 Delete Directory 25
8.3 File Operations **26**
8.3.1 Get File 26
8.3.2 Create File 26
8.3.3 Update File 26
8.3.4 Delete File 26
8.4 File Data Transfer **27**
8.5 Current File-System Limitations **28**
9. Synchronisation Model ***28***
9.1 Model Definition **28**
9.2 Supported Current Behaviour **28**
9.3 Unsupported Current Behaviour **29**
9.4 Future Orchestrated and Heuristic Sync Direction **29**
10. Security Implementation ***30***
10.1 Security Position **30**
10.2 Cryptographic Primitives **30**
10.3 Transport Payload Encryption **30**
10.4 First-Time Registration Security **31**
10.5 Registered Client Authentication **31**
10.6 Key Management **32**
10.7 OS-Level Isolation **33**
10.8 SFTP Jail Model **33**

10.9 Shell Command Trust Boundary 34
11. Client Connection Flows..... 34
11.1 Local First-Time Registration Flow 34
11.2 Registered Local Command Flow 36
11.3 Remote PathFinder Command Flow 36
11.4 Local-Only Multi-Client Registration 37
12. Networking Interface 38
12.1 Local Discovery 38
12.2 Setup AP Networking 38
12.3 Local Managed TCP Channel 41
12.4 Remote PathFinder WebSocket Channel 41
12.5 Managed UDP Availability 43
12.6 Heartbeat & Disconnection Model 43
12.7 Trust Model 44
13. API/Interface Definitions 44
13.1 Native Request Packet 44
13.2 Native Response Packet 44
13.3 Command Dispatch Semantics 45
13.4 Response Payload Pattern 45
14. Command Catalogue 46
14.1 Client Management Commands 46
14.2 Wi-Fi Commands 47
14.3 File-System Commands 47
15. Wi-Fi Management 48
15.1 Wi-Fi Scope 48
15.2 Setup AP Mode 48
15.3 Wi-Fi Listing 49
15.4 Wi-Fi Connection 49
15.5 Current Wi-Fi Detection 50
15.6 Forget Current Wi-Fi 50
15.7 Wi-Fi Limitations 51
16. Update System 52
16.1 Release Channels 52
16.2 Metadata Sources 52
16.3 Update Check Algorithm 53

16.4 Binary Replacement **53**
16.5 Restart Behaviour **53**
16.6 Update Integrity Requirements **54**
16.7 Update Limitations **54**
17. Deployment & Runtime Model ***54***
17.1 Target Hardware **54**
17.2 Operating System **54**
17.3 Runtime Environment **55**
17.4 Host Dependencies **55**
17.5 Startup Service **55**
17.6 Resource Manager Service Position **56**
17.7 Platform Scope **57**
18. Performance & Resource Constraints ***57***
18.1 Runtime Characteristics **57**
18.2 Network Constraints **57**
18.3 File-System Constraints **58**
18.4 Update Polling **58**
18.5 Resource Risks **58**
19. Logging & Observability ***59***
19.1 Logging Location **59**
19.2 Severity Levels **59**
19.3 Logged Events **59**
19.4 Current Observability Limitations **61**
19.5 Future Metrics **61**
20. Failure Modes & Recovery ***61***
20.1 Configuration Failure **61**
20.2 Network Failure **62**
20.3 PathFinder Failure **63**
20.4 Update Failure **63**
20.5 File-System Failure **64**
20.6 SFTP/OS User Failure **64**
20.7 Command Failure **64**
21. Testing & Validation ***65***
21.1 Current State **65**
21.2 Required Unit Tests **66**

21.3 Required Integration Tests **67**
21.4 Runtime Validation **69**
21.5 Acceptance Criteria **69**
22. Risks & Limitations ***71***
22.1 Technical Risks **71**
22.2 Security Risks **71**
22.3 Reliability Risks **72**
22.4 Operational Risks **72**
22.5 Protocol Compatibility Risks **73**
Appendices ***74***
Appendix A: Important Constants **74**
Appendix B: Glossary **77**
Appendix C: Acronyms **78**
Appendix D: Component Diagram **79**
Appendix E: Boot Sequence Diagram **80**
Appendix F: Command Handling Diagram **81**
Appendix G: Local Registration Flow **82**
Appendix H: Remote PathFinder Flow **82**

Version: V1.0.0 (Major, Minor, Patch)
Status: ON-GOING
Owner: Steve Yoo
Last Updated: 09/11/2026

---

1. Overview

Roof Engine is the core runtime and orchestration layer of a Roof device.

It is responsible for managing the device-side execution environment, coordinating registered clients, handling local and remote command traffic, maintaining the shared Roof storage boundary, enforcing security controls, and bridging local-first Roof operation with wider remote routing through Roof PathFinder.

Roof Engine is not a user-facing application. It operates as the system-level runtime process behind RoofDeck, Roof.Client, PathFinder-routed access, file-system operations, device configuration, and future Roof-compatible client applications.

At a high level, Roof Engine acts as:

- the runtime process of a Roof device;
- the device-side command execution layer;
- the local networking endpoint for Roof clients;
- the remote networking endpoint for PathFinder-routed clients;
- the shared file-system manager;
- the security and credential coordination layer;
- the system update and maintenance process.

Roof Engine is currently implemented as a .NET 10 NativeAOT executable targeting Linux ARM64 environments, especially Raspberry Pi-based Roof hardware. It is strongly tied to Linux system services, OpenSSH/SFTP, systemd, nmcli, shell utilities, and per-client Linux user isolation.

1.1 Purpose

The purpose of Roof Engine is to provide the core runtime layer that allows a Roof device to operate as a local-first, self-hosted, user-controlled data and communication node.

Roof Engine supports the wider Roof infrastructure by:

- accepting local client connections;
- registering and authenticating Roof clients;
- executing encrypted command packets;
- exposing controlled file-system operations;
- managing shared storage accessible to registered Roof client accounts;
- configuring local device networking where appropriate;
- connecting to Roof PathFinder for remote relay communication;
- performing update checks and binary replacement;

- supporting the operational foundation for RoofDeck and future Roof-compatible applications.

1.2 Scope

This specification covers:

- Roof Engine responsibilities;
- runtime startup and service lifecycle;
- local and remote networking behaviour;
- command packet handling;
- file-system storage model;
- configuration model;
- client state model;
- security and encryption behaviour;
- SFTP / Linux user isolation;
- update and deployment model;
- logging and observability;
- known implementation gaps;
- testing and validation requirements;
- risks, limitations, and future design decisions.

This specification does NOT fully define:

- RoofDeck UI behaviour;
- Roof.Client host application behaviour;
- PathFinder relay internals except where Engine integration requires it;
- complete future RApp SDK behaviour;
- final user onboarding UX;
- final production security policy;
- final continuous file sync strategy;
- final multi-device conflict resolution policy.

1.3 Role within the Roof Ecosystem

Roof Engine interfaces with the following Roof components:

Component	Relationship
RoofDeck	User-facing client application that discovers, registers with, and sends commands to Roof Engine.
Roof.Client	Internal client-side library used by applications to communicate with Roof Engine.
Roof.Common	Shared protocol, networking, encryption, logging, DTO, and JSON serialisation library.

Component	Relationship
Roof PathFinder	Remote relay used when local direct communication is unavailable or unsuitable.
Roof Hardware	Linux-based physical device environment where Engine runs.
Future RApps / SDK consumers	Future client applications that may communicate with Roof Engine through standardised client APIs.

Roof Engine is the device-side authority for command execution, client storage, and local runtime behaviour. PathFinder may relay encrypted packets, but it does not execute Roof commands or own Roof data.

2. Functional Responsibilities

Roof Engine is responsible for operating the device-side Roof runtime.

2.1 Core Responsibilities

2.1.1 System Lifecycle

Roof Engine must:

- start as the main Roof runtime process;
- log version and compile channel information;
- perform security and integrity checks;
- perform device checks;
- check internet availability;
- initialise configuration;
- initialise runtime services;
- keep the runtime alive;
- support update and restart workflows.
- In IA, Beta and Release builds, attempt to create and start the Setup AP during startup when internet connectivity is unavailable.
- Continue direct local-network operation if Setup AP creation or activation fails.
- Monitor internet connectivity and Setup AP state during runtime at 15-second intervals.
- Attempt to stop an active Setup AP when internet connectivity returns, and attempt to start an inactive Setup AP when internet connectivity is unavailable.

Initial setup completion is not an independent Setup AP start or stop condition in the current implementation. During startup, HasCompletedInitialSetup selects the diagnostic message. Internet availability determines whether Setup AP creation is attempted.

2.1.2 Client Management

Roof Engine must:

- accept local TCP clients;

- respond to local discovery requests;
- support first-time registration;
- maintain connected client state;
- distinguish local and remote clients;
- identify registered clients;
- dispatch client commands;
- send command responses over the correct channel;
- maintain per-client access counts;
- associate registered clients with OS-level usernames.

2.1.3 File-System Management

Roof Engine must:

- maintain the shared storage root at /srv/roof/playground
- build in-memory file and directory metadata;
- expose command-driven file and directory operations;
- maintain hashed path identifiers;
- update file hashes and timestamps;
- constrain Engine-mediated filesystem operations to the shared Roof playground at /srv/roof/playground;
- interact with Linux ownership and permission controls.

2.1.4 Networking

Roof Engine must handle:

- UDP local presence discovery;
- managed TCP local command traffic;
- WebSocket remote command traffic through PathFinder;
- Setup AP networking for local onboarding and recovery when internet connectivity is unavailable, subject to wireless hardware and NetworkManager availability;
- heartbeat and timeout handling;
- local Wi-Fi listing and connection commands;
- remote client heartbeat forwarding through PathFinder.
- maintain stable PathFinder WebSocket reconnection behaviour using fixed reconnect delay and jitter;
- clean up failed PathFinder provider initialisation attempts before retrying;
- detect WebSocket close and non-open socket states as disconnection events;
- avoid tight heartbeat and reconnection loops that can consume a CPU core;
- support safer local Wi-Fi connection creation through Roof-owned NetworkManager profiles;
- avoid active macOS subnet ping sweeps during local-device discovery.

Roof Engine must support heuristic Setup AP management.

Setup AP startup is no longer limited only to HasCompletedInitialSetup == false. In supported release-like builds, Engine may also create and start the Setup AP when internet connectivity is unavailable, allowing local recovery or direct device access when the device cannot reach external networks.

PathFinder connectivity now uses a fixed reconnect delay model rather than exponential backoff. Failed provider initialisation must be cleaned up before the next attempt, and the provider must only be published to ClientManager after successful initialisation. WebSocket disconnection detection must treat non-open socket state, close frames, and WebSocket exceptions as client-disconnected events.

2.1.5 Security

Roof Engine must:

- encrypt command payloads after registration;
- manage AES network keys and IVs;
- validate packet decryption;
- reject malformed packets;
- constrain filesystem access to the shared Roof playground;
- manage Linux user and SFTP boundaries;
- support credential rotation commands;
- validate update binary integrity.

2.1.6 Updates & Maintenance

Roof Engine must:

- identify current build channel;
- check release metadata;
- compare version and binary hash;
- download updated Engine binaries;
- replace the running binary where required;
- restart or re-execute the Engine process;
- avoid applying invalid or mismatched update artifacts.

2.2 Out-of-Scope Responsibilities

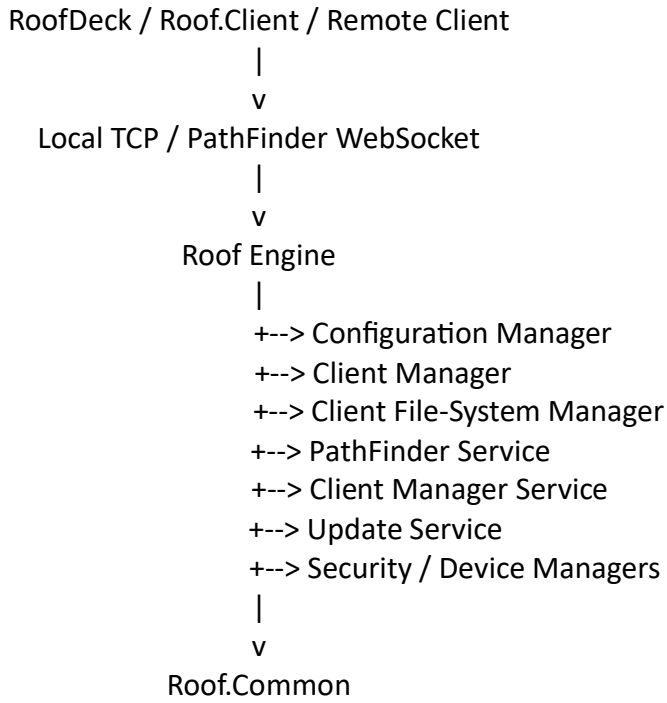
Roof Engine should NOT be treated as:

- a public cloud storage service;
- a central data owner;
- a PathFinder replacement;
- a general-purpose web server;
- an application marketplace;
- a UI layer;
- a permanent telemetry aggregation service;
- a complete conflict-resolution engine in its current form;
- a real-time background file sync daemon in its current form.

3. System Architecture

3.1 High-Level Architecture

Roof Engine follows a modular service-based architecture.
At a high level:



Roof Engine depends on Roof.Common for:

- packet models;
- JSON serialisation contexts;
- RNMS networking providers;
- AES encryption helpers;
- SHA-256 and xxHash64 hashing;
- logging;
- file and directory DTOs;
- local network discovery support.

3.2 Process Startup

Program.Main controls the Engine startup process.

Startup behaviour:

1. Log version, compile flag, and channel-specific informational version.
2. Run security and integrity checks.
3. Run device checks.
4. Check internet availability using ICMP ping.
5. In release-like channels, perform update check if internet is available.
6. Ensure OpenSSH/SFTP support exists and is configured.
7. In release-like channels, ensure startup systemd service exists.
8. Initialise configuration.
9. Load active configuration.
10. Initialise registered services.
11. Start registered services as background threads.
12. Enter the main keepalive loop.

PathFinder provider initialisation must be treated as conditional. If a provider fails to initialise, Engine must detach handlers, dispose the failed provider, log the failure as a warning, and continue local operation while retrying PathFinder connection later.

3.3 Runtime Service Model

Roof Engine services inherit from a common service base contract:

- Initialize()
- Start()
- Terminate()

Services are registered through ServiceManager.
Current registered services include:

Service	Responsibility
ClientManagerService	Provides UDP presence discovery, the local managed TCP listener, local client lifecycle handling, and continuous Setup AP requirement monitoring.
PathFinderService	Remote WebSocket connection lifecycle and PathFinder-routed clients.
UpdateService	Periodic update checks in release-like channels.
ResourceManagerService.	Runs a cancellable 60-second PeriodicTimer loop that invokes log rotation and compressed-log eviction through SystemResourceManager.

3.4 Main Modules

Module	Responsibility
Program	Entry point, startup checks, systemd setup, SFTP setup, service lifecycle.
ConfigurationManager	Config creation, loading, saving, migration, and hash generation.
ClientManager	Client state, registration, packet decryption, command dispatch, responses.
ClientFileSystemManager	Shared filesystem metadata, path validation and physical file operations within /srv/roof/playground. Client filesystem manager instances use shared storage and shared metadata caches.

Module	Responsibility
ClientManagerService	UDP presence and local TCP command listener.
PathFinderService	Remote WebSocket connection, remote client tracking, heartbeat handling.
UpdateService	Background update polling.
UpdateManager	Version/hash comparison, binary download, replacement, restart.
ReleaseServerIngressManager	Release metadata retrieval and artifact URL construction.
SecurityManager	Security/integrity check placeholder.
DeviceManager	Device check placeholder.
SystemResourceManager	Performs log-file rotation, compression, and retention cleanup. Legacy Linux service-disabling helpers remain present but are not currently invoked by ResourceManagerService.
Shell	Async /bin/bash -c command runner.
WifiAPLister	Parser and ranker for nmcli Wi-Fi output.
SetupAPManager	Serialises Setup AP lifecycle operations; checks wireless readiness; creates and validates the RoofAP NetworkManager profile; ensures the ap0 interface exists and is active; starts and stops the AP; and performs guarded wireless recovery after activation failures.
ShutdownManager	Coordinates shutdown signals, tracked client work, service termination, configuration saving, and deferred reboot.

3.5 Roof.Common Dependency

Roof.Common provides shared support functionality used by Roof Engine, Roof.Client, and PathFinder.

In the Engine context, Roof.Common provides:

- RoofNetworkNativeRequestPacket;
- RoofNetworkNativeResponsePacket;
- RoofWebSocketPacket;
- file and directory metadata DTOs;

- JSON source-generation contexts for NativeAOT;
- RNMS managed TCP, managed UDP, unmanaged UDP, and WebSocket providers;
- AES-256 encryption helpers;
- SHA-256 and xxHash64 hashing helpers;
- logging utilities;
- network discovery utilities.

4. Runtime Lifecycle

4.1 Boot Sequence

The observed Engine boot sequence is:

1. Register shutdown signal and process-exit handlers.
2. Log the Engine version and compile configuration.
3. Invoke the security/integrity and device checks. These checks currently log messages rather than perform substantive validation.
4. Check internet availability using an ICMP ping to 8.8.8.8.
5. In IA, Beta and Release builds, attempt an update check when internet connectivity is available.
6. Check and configure OpenSSH/SFTP support.
7. In IA, Beta and Release builds, ensure the startup systemd unit exists and is enabled.
8. Initialise ConfigurationManager, creating or loading the configuration and enforcing the applicable decryption, integrity and migration checks.
9. In IA, Beta and Release builds, attempt Setup AP creation and activation when internet connectivity is unavailable. Setup AP failure is logged and does not, by itself, prevent direct local setup.
10. If registered clients have OS usernames, migrate legacy per-user playground contents into the shared playground.
11. Provision the shared playground, SFTP group, permissions, default ACLs and shared SFTP configuration. This provisioning also runs on fresh installations.
12. Restart SSH. Failure during shared playground provisioning or this SSH restart prevents service startup.
13. Initialise and start ClientManagerService, PathFinderService, UpdateService and ResourceManagerService.
14. Wait indefinitely using the shutdown cancellation token, then enter coordinated shutdown when cancellation is requested.

HasCompletedInitialSetup is not an independent startup AP trigger. An incomplete setup with working internet connectivity does not, by itself, cause startup AP creation.

ClientManagerService evaluates Setup AP requirements every 15 seconds. This runtime monitor also starts in Debug builds. It stops an active AP when internet connectivity returns and attempts to start an inactive AP when connectivity is unavailable.

PathFinder connection establishment runs in the background. Failed initialisation attempts are cleaned up, and the service retries without preventing local Engine operation.

4.2 System Checks

Current system checks include:

- security/integrity placeholder check;
- device placeholder check;
- internet availability check;
- release update check;
- OpenSSH/SFTP availability;
- systemd startup service creation;
- configuration initialisation.

Setup networking checks include:

- checking internet availability;
- checking whether NetworkManager tooling is available during periodic monitoring;
- detecting whether the RoofAP connection is active on ap0;
- checking wlan0 presence, Wi-Fi radio state, rfkill state and interface readiness;
- ensuring ap0 exists and is brought online;
- validating or recreating the RoofAP connection profile during activation;
- attempting guarded wireless recovery after setup or activation failure;
- allowing direct local setup to continue when AP creation or activation fails; and
- stopping the AP when internet connectivity returns.

Initial setup completion is retained as configuration state, but it is not currently used as an independent AP lifecycle condition.

The security and device checks currently are placeholders and should be expanded into concrete checks before production hardening.

4.3 Service Initialisation

ServiceManager.InitializeServices() initialises all registered services.

Each service is expected to complete its own setup before the Engine enters normal runtime operation.

Service initialisation failure handling should be defined more strictly in future versions. A failed core service should either block startup or place the Engine into a degraded-but-explicit state.

ServiceManager currently initialises and starts ClientManagerService, PathFinderService, UpdateService, and ResourceManagerService.

ClientManagerService starts three background activities:

- UDP presence-beam handling;
- local managed TCP connection handling;
- Setup AP requirement monitoring.

ResourceManagerService starts a cancellable periodic task for log-file lifecycle management.

4.4 Main Loop

After service startup, Engine awaits an infinite task using the shutdown cancellation token. Cancellation releases the main wait and enters the shutdown sequence.

Runtime work occurs in background service threads and asynchronous network callbacks.

Background loops must not spin continuously. Runtime loops that wait for PathFinder state, heartbeat opportunities, or update intervals must use explicit delays, blocking receive calls, or timers. Tight polling loops are not acceptable because they can cause one CPU core to remain permanently saturated.

4.5 Shutdown Behaviour

Roof Engine implements coordinated shutdown through ShutdownManager.

Shutdown can be requested by Ctrl+C, SIGTERM, process exit, an unhandled exception, an observed background-task failure or a device restart request. Program.Main invokes the shared shutdown sequence from its finally block.

Services terminate in this order:

1. UpdateService.
2. ResourceManagerService.
3. ClientManagerService.
4. PathFinderService.

Each service termination has a ten-second timeout. Local client handling is stopped, local network resources are disposed, and the PathFinder provider and remote client state are cleaned up through service termination.

After service termination, Engine waits up to ten seconds for tracked client work and attempts to save the loaded configuration. It then logs shutdown completion and flushes console output. File-log writers are disposed after each entry.

Process-exit callbacks wait at most sixty seconds for shutdown. Timeout and failure paths set a nonzero process exit code where implemented.

When a reboot has been requested, ShutdownManager attempts sudo reboot after the shutdown sequence. The current implementation still attempts reboot after recorded shutdown failures; it does not require a completely successful shutdown as a reboot precondition.

Timeout handling, active-request disposal ordering and forced termination remain validation areas.

5. Client State Model

5.1 Runtime Client Object

A runtime RoofClient represents a connected or partially connected client.

It stores:

- ClientID;
- local endpoint for TCP clients;
- PathFinder routed fingerprint for remote clients;
- remote/local mode state;
- verification state;
- first-time encryption exchange state;
- nonce state;
- file-system manager;
- PathFinder heartbeat timestamps;
- cached encryption information.
- ConnectedTime
- RegistrationClientID
- IsAdditionalRegistration
- IsDisposed

Each client object provides a state lock and a semaphore for serialising its command execution.

5.2 Client Collections

ClientManager maintains process-wide client collections:

Collection	Purpose
ConnectedClients	Clients currently recognised by Engine.
FirstTimeUnverifiedClients	Clients in first-time setup or encryption exchange state.

Local TCP clients are identified by endpoint.

Remote clients are identified by:

- ClientID; and
- PathFinder-provided sender fingerprint.

Unverified client state older than two minutes is removed during periodic cleanup.

Successful authentication removes previous sessions with the same client ID and the same local/remote mode.

5.3 Client Verification State

Client state includes multiple verification-related fields:

- whether first-time TCP encryption info has been sent;

- whether a client is verified;
- whether nonce verification has completed;
- whether the client is in remote mode.

The intended verification model is stronger than the currently active implementation. The current registered-client path mainly relies on successful decryption with stored AES credentials.

5.4 Encryption Info Lookup

RoofClient.GetEncryptionInfo() resolves encryption credentials in this order:

1. first-time temporary key and IV;
2. cached key and IV;
3. registered client credentials read from /etc/Roof/keys/{clientId}.key;
4. null values if no credentials are available.

This allows first-time setup traffic to use temporary session keys and registered traffic to use persisted client credentials.

Disposed clients return no encryption credentials. Disposal clears temporary credentials and the cached key/IV pair.

6. Configuration Model

6.1 Configuration File

Configuration is stored at: {engine working directory}/roof_config/config.json
The current Engine spec may refer to /roof_config/config.json, but the observed implementation uses a path relative to the Engine working directory.

6.2 Configuration Schema

Current schema version: 2. The root configuration object contains:

Field	Meaning
Sha256FileHash	Stored hash of serialised config with the hash field excluded.
SchemaVersion	Config schema version.
RoofIdentifier	Engine/device identifier, generated as a GUID without hyphens.
HasCompletedInitialSetup	Indicates whether first setup has completed.
RegisteredClients	Dictionary of registered client configuration entries keyed by client ID.

6.3 Registered Client Configuration

Each registered client entry contains:

Field	Meaning
AccessCount	Count of client access/connect events.
OSSystemUsername	Linux username mapped to the client.

Current per-client network encryption credentials are stored at:
/etc/Roof/keys/{clientId}.key

Each client key file contains:

- 32 bytes of AES-256 key material;
- followed by 16 bytes of AES IV material.

Engine applies file mode 600 to each client key file.

The keys directory is secured with root:root ownership and mode 700. Key files use root:root ownership and mode 600.

6.4 Config Save Behaviour

The current save algorithm is:

1. Reject null config.
2. Set the loaded configuration reference after successful file replacement.
3. Temporarily exclude Sha256FileHash and compute a SHA-256 integrity hash over the canonical serialised configuration.
4. Store the computed hash in Sha256FileHash.
5. Serialise the configuration using the source-generated JSON context.
6. Read the configuration encryption key from /etc/Roof/keys/roof_config.key.
7. Validate that the key file contains exactly 48 bytes.
8. Interpret the first 32 bytes as the AES-256 key and the following 16 bytes as the AES IV.
9. Encrypt the serialised configuration.
10. Write encrypted configuration to a uniquely named temporary file, then replace config.json by rename.

Per-client network encryption keys and IVs are not serialised into the current configuration schema. They are persisted separately in protected client key files.

Sensitive values must not be written to disk as plaintext.
This produces a tamper-detection field, but enforcement depends on load-time validation.

6.5 Config Load Behaviour

The current load algorithm is:

1. Read the persisted configuration file.
2. Read /etc/Roof/keys/roof_config.key and require a total length of 48 bytes.

3. Interpret the first 32 bytes as the AES key and the following 16 bytes as the IV.
4. Decrypt the configuration before parsing JSON.
5. Deserialise the configuration.
6. Require a stored Sha256FileHash.
7. Validate the stored hash against the configuration content. Older schemas use the legacy integrity calculation before migration.
8. For an older supported schema, migrate legacy fields and client credentials, update the schema version to 2, and save the migrated configuration in encrypted form.
9. Return the validated configuration to the initialisation path.

Missing or invalid key material, decryption failure, invalid JSON, integrity failure or migration failure blocks normal startup.

The loader does not currently accept a raw plaintext configuration file.

The configuration encryption key is stored at /etc/Roof/keys/roof_config.key. The file contains a 32-byte AES-256 key followed by a 16-byte IV and must have a total length of 48 bytes.

If the configuration key file is missing or has an invalid length, configuration loading fails and normal Engine startup is blocked.

6.6 Config Migration

Migration supports legacy field mappings:

Legacy Field	New Field
NetworkEncryptionKey	First 32 bytes of /etc/Roof/keys/{clientId}.key after Base64 decoding.
NetworkEncryptionIV	Following 16 bytes of /etc/Roof/keys/{clientId}.key after Base64 decoding.
SystemUsername	OSSystemUsername
AccessCount	retained

The current loader migrates older encrypted configuration schemas. It does not provide a raw plaintext configuration fallback.

Plaintext-to-encrypted configuration migration is a future requirement and is not implemented by the current loader.

Before this capability is enabled, its implementation must:

- identify an eligible legacy plaintext configuration;
- validate its structure and available integrity information;

- preserve registered clients, credentials, OS username mappings, setup state and RoofIdentifier;
- persist client credentials in the protected client key files;
- write the migrated configuration in encrypted form; and
- fail without replacing the original configuration when migration cannot be completed.

Current migration support applies to older encrypted configuration schemas.

During migration, Engine examines each legacy RegisteredClients entry for NetworkEncryptionKey and NetworkEncryptionIV. When both values are available, Engine Base64-decodes them and creates the client-specific key file.

Migration upgrades the configuration to schema version 2 and saves the migrated configuration in encrypted form.

Legacy credential fields may still be recognised as migration inputs, but they are not part of the current RoofClientConfig data model.

Older schemas are integrity-checked using their legacy serialied fields before migration. Invalid legacy credentials or failed key/configuration writes abort migration.

6.7 Config Security Requirements

Roof Engine configuration must be encrypted at rest.

Configuration contains sensitive operational values including registered client records, client-to-system mappings, device identity, and setup state. The configuration is encrypted at rest.

Per-client AES network keys and IVs are stored separately in protected binary files under /etc/Roof/keys. They are not fields in the current registered-client configuration object.

Required behaviour:

- sensitive configuration data must be encrypted at rest;
- client encryption credentials must not be exposed in plaintext;
- encryption key material must be separated from the plaintext configuration file;
- Engine must fail safely where configuration cannot be decrypted;
- configuration encryption must be implemented before production maturity.

Configuration integrity enforcement is also mandatory.

If the stored configuration hash does not match the computed configuration hash during load, Roof Engine must treat the configuration as tampered, corrupted, or invalid.

Required integrity behaviour:

- validate configuration integrity during startup;
- compare stored configuration hash against computed configuration hash;

- refuse normal startup when configuration integrity validation fails;
- log the failure as a critical security event;
- support a controlled recovery path where appropriate.

Current implementation status:

- configuration encryption is implemented;
- configuration schema version 2 is active;
- configuration encryption material is stored in /etc/Roof/keys/roof_config.key;
- per-client network credentials are stored in /etc/Roof/keys/{clientId}.key;
- key files contain binary key and IV material rather than Base64 JSON properties;
- key-file permissions are set to mode 600;
- legacy client key and IV fields are migrated into separate key files;
- configuration integrity enforcement is enabled during load;
- tampered, corrupt, unreadable, or invalid configurations block normal startup.

7. Data Model & Storage

7.1 Shared Storage Root

Each client is mapped to a Linux user. All client filesystem managers operate on /srv/roof/playground. Registered client accounts retain separate identities and credentials but share the same stored files.

7.2 File-System Metadata Model

Roof Engine maintains shared in-memory file and directory metadata used by all client filesystem managers. The metadata model uses hashed path identifiers rather than exposing raw paths as the main command identifier.

Hashing method: xxHash64(path)
Hashed path output format: 16-character hexadecimal string

7.3 Directory Metadata

RoofDirectoryData contains:

Field	Meaning
HashedPath	xxHash64 of the directory path.
Name	Directory name.
Path	Physical directory path.
Size	Recursive directory size.
LastModifiedUTC	Last modified timestamp.
Files	Set of child file hashed paths.

Field	Meaning
SubDirectories	Set of child directory hashed paths.
ParentDirectory	Parent directory hashed path.

7.4 File Metadata

RoofFileData contains:

Field	Meaning
HashedPath	xxHash64 of the file path.
Name	File name.
Path	Physical file path.
Size	File size.
FileHash	xxHash64 of file content.
LastModifiedUTC	Last modified timestamp.
ParentDirectory	Parent directory hashed path.
Data	Raw payload bytes used for file transfer.
ShouldOverwriteData	Indicates overwrite write mode.
ShouldAppendData	Indicates append write mode.

CompressData() and DecompressData() currently exist as placeholders and do not implement active compression.

7.5 Indexing Strategy

ClientFileSystemManager builds in-memory dictionaries for:

- top-level directories;
- subdirectories;
- top-level files;
- subfiles.

The shared base index is constructed when a client filesystem manager is created and the shared base directory has not already been loaded.

The manager performs:

- base directory initialisation;
- immediate child scan;

- breadth-first directory traversal;
- recursive directory size calculation;
- hashed path generation;
- metadata dictionary population.

7.6 Data Consistency Boundaries

The Engine file-system cache reflects:

- state at initial scan time; and
- updates made through Engine-mediated commands.

External changes made directly on the filesystem are not continuously watched.

Current consistency caveats:

- no continuous file watcher;
- no background reconciliation;
- missing physical files may return null but are not always cleaned from cache;
- file timestamps are exposed but not used for conflict resolution.

8. File-System Behaviour

8.1 Initial Index Construction

When ClientFileSystemManager is constructed:

1. _basePath is set to /srv/roof/playground.
2. _currentWorkingDirectory is set to the base path.
3. LoadBaseDirectory() constructs the base directory metadata.
4. The base directory is named Home.
5. Immediate child directories and files are loaded.
6. Child directories are scanned breadth-first.
7. Child links are stored as hashed path strings.
8. Directory and file objects are cached in dictionaries.

The base directory must not be deleted through file-system commands.

8.2 Directory Operations

8.2.1 Get Directory

GET_DIRECTORY resolves a directory by hashed path.

Lookup order:

1. base directory;
2. top-level directory cache;
3. subdirectory cache.

Before returning, the Engine refreshes LastModifiedUTC from the physical directory where possible.

8.2.2 Create Directory

CREATE_DIRECTORY:

1. deserialises RoofDirectoryData;
2. creates the physical directory;
3. applies Linux ownership;
4. applies directory permissions;
5. finds the parent directory;
6. links the new directory into parent metadata;
7. adds the new directory to the correct cache.

Expected ownership command: sudo chown root:roofsftpusers -- "{path}"
Expected permission command: sudo chmod 2770 -- "{path}"

8.2.3 Update Directory

UPDATE_DIRECTORY:

1. finds existing metadata by old hashed path;
2. compares updated metadata;
3. updates name, path, parent, and size where required;
4. updates parent-child links where parent changes;
5. updates cache key where hashed path changes;
6. physically moves the directory when required.

Directory moves update the physical tree and rebuild cached descendant paths, hashes, parent identifiers, and child links. Verification failure triggers an attempt to restore the original physical location.

8.2.4 Copy Directory

COPY_DIRECTORY:

1. finds source directory;
2. finds target directory;
3. creates new metadata under the target;
4. links new directory under target metadata;
5. recursively copies physical directory content.

Copy operations recursively construct metadata for copied directories and files before publishing cache entries. Failed copies trigger cleanup of the newly created destination.

8.2.5 Delete Directory

DELETE_DIRECTORY:

1. rejects deletion of base directory;
2. finds target directory;
3. unlinks target from parent metadata;
4. removes target from caches;
5. deletes the physical directory recursively;
6. verifies deletion.

8.3 File Operations

8.3.1 Get File

GET_FILE:

1. finds file by hashed path;
2. verifies the physical file exists;
3. computes or refreshes file hash if content exists;
4. refreshes last modified timestamp;
5. returns metadata.

8.3.2 Create File

CREATE_FILE:

1. deserialises RoofFileData;
2. refuses creation if the physical file already exists;
3. creates an empty physical file;
4. applies Linux ownership;
5. applies permissions;
6. links file metadata under parent directory;
7. adds file metadata to cache.

Important behaviour:

- CREATE_FILE creates an empty file;
- actual content is expected to arrive through UPDATE_FILE.

8.3.3 Update File

UPDATE_FILE:

1. finds existing file by hashed path;
2. validates file payload if supplied;
3. computes xxHash64 over incoming data;
4. compares incoming hash to existing file hash;
5. validates write flags;
6. overwrites or appends content;
7. updates hash and size;
8. returns updated file metadata.

Write flag rule:

- exactly one of ShouldOverwriteData or ShouldAppendData should be true when writing new content.

File updates compare incoming name, path, and parent values against existing metadata. A successful physical move precedes publication of the new cached identity.

Append executes even when the supplied bytes equal existing content. The final hash and size are calculated from the resulting complete file.

8.3.4 Delete File

DELETE_FILE:

1. finds file metadata;
2. finds parent directory;
3. unlinks file from parent metadata;
4. removes file from cache;
5. deletes physical file;
6. verifies physical deletion.

8.4 File Data Transfer

GET_FILE_DATA currently:

1. finds file by hashed path;
2. verifies physical file exists;
3. reads all bytes;
4. returns Base64 file content in a single response packet.

Current behaviour:

- file data is returned as a single Base64 payload;
- chunked file transfer is not currently implemented;
- compression is not currently implemented;
- the current transfer model must not be represented as chunked, resumable, or compressed transfer.

Future chunked file transfer is accepted as a required capability, especially for:

- larger files;
- unreliable networks;
- PathFinder-routed transfers;
- memory-constrained hardware;
- resumable transfer behaviour.

Before chunked file transfer is implemented, the following must be defined:

- chunk size;
- transfer session ID;
- sequence numbering;
- retry behaviour;
- resume behaviour;
- cancellation behaviour;
- final file hash validation;
- partial transfer cleanup;
- transfer completion acknowledgement;
- behaviour differences between local TCP and PathFinder-routed transfer;
- user-tested file-size expectations.

Future compression is also accepted as a required capability where it provides practical benefit.

Before compression is implemented, the following must be defined:

- compression algorithm;

- compression threshold;
- whether compression is automatic or client-requested;
- whether compression occurs before or after encryption;
- CPU impact on Raspberry Pi hardware;
- memory impact on Raspberry Pi hardware;
- compatibility with chunked transfer;
- user-tested workload assumptions.

Current implementation status:

- chunked transfer is currently unimplemented until further decision and user testing;
- compression is currently unimplemented until further decision and user testing;
- existing DTO compression and decompression methods are placeholders only.

8.5 Current File-System Limitations

Current limitations include:

- no continuous file watching;
- no background reconciliation;
- no explicit conflict resolution;
- no active chunking for large file data;
- compression/decompression methods are placeholders;
- external file changes may not be reflected until explicit access or restart;
- a process-wide semaphore protects Engine filesystem operations and shared caches. External SFTP writes remain outside this synchronisation.

9. Synchronisation Model

9.1 Model Definition

Roof Engine currently implements a deterministic, command-driven file-system operation model. It does not currently implement a continuous autonomous background sync engine.

All supported file and directory operations are explicitly triggered by client command packets.

The current Engine must not behave as:

- a hidden background sync daemon;
- an automatic reconciliation service;
- an autonomous conflict resolver;
- a background file mutation service.

9.2 Supported Current Behaviour

Supported command-driven operations include:

- get base directory;
- get directory;
- create directory;
- update directory;

- copy directory;
- delete directory;
- get file metadata;
- create file;
- update file;
- delete file;
- get file data as Base64.

Startup migration of legacy playgrounds is a maintenance exception to command-triggered filesystem mutation.

Current behaviour must remain deterministic:

- file and directory mutations occur only when explicitly commanded;
- Engine-mediated operations should produce predictable results;
- hidden background mutation should be avoided;
- conflict resolution should not occur autonomously in the current implementation.

9.3 Unsupported Current Behaviour

Currently unsupported or incomplete behaviours include:

- real-time background sync;
- background file watching;
- automatic reconciliation;
- autonomous conflict resolution;
- active chunked file transfer;
- compression;
- GET_DIRECTORIES implementation;
- offline sync queues;
- merge policies.

9.4 Future Orchestrated and Heuristic Sync Direction

Future synchronisation may become orchestrated and heuristic.

This future model may include:

- higher-level sync orchestration;
- hash and timestamp comparison;
- client-side and Engine-side state reconciliation;
- user-intent-aware conflict handling;
- partial sync recovery;
- local-first priority rules;
- explicit sync sessions;
- heuristic conflict detection;
- structured reconciliation workflows.

Future orchestrated and heuristic sync behaviour must be specified separately before implementation.

Until then, Roof Engine must remain command-driven.

10. Security Implementation

10.1 Security Position

Roof Engine is a trusted local runtime process on the user's Roof device.

It must protect:

- client credentials;
- command payloads;
- configuration secrets;
- the shared Roof playground and its access boundaries;
- update integrity;
- OS-level boundaries;
- remote routed communication confidentiality.

The strongest current security boundaries are:

- AES-encrypted command payloads;
- per-client Linux users;
- SFTP jail configuration;
- file ownership and permissions;
- registered-client encryption credentials;
- update hash validation.

10.2 Cryptographic Primitives

Current cryptographic primitives include:

Primitive	Use
AES-256	Network payload encryption.
PKCS7 padding	AES block padding.
Base64	Transport-safe ciphertext/key/IV representation.
SHA-256	Config hash and update binary validation.
xxHash64	Hashed path IDs and file content hashes.
RandomNumberGenerator	Nonce, password, key-related randomness.
MD5	Utility helper exists but is not core to Engine security.

10.3 Transport Payload Encryption

Network encryption uses AES-256.

Behaviour:

- AES keys are Base64 strings;
- keys decode to 32 bytes;
- IVs decode to 16 bytes;
- plaintext packet JSON is UTF-8 encoded;
- ciphertext is Base64 encoded;
- network payload is UTF-8 bytes containing the Base64 ciphertext;
- AES uses PKCS7 padding.

After registration, command traffic is expected to be encrypted.

10.4 First-Time Registration Security

First-time registration currently follows this model:

1. Client sends unencrypted JSON request with TCP INFO REQUEST.
2. Engine generates AES key and IV.
3. Engine sends TCP_ENCRYPTION_INFO, client ID, key, and IV over local TCP.
4. Client sends encrypted REGISTER REQUEST.
5. Engine stores the registered-client record in configuration and network credentials in the client-specific key file.
6. Engine marks initial setup complete.
7. Engine returns encrypted registration success.

Security caveat:

- the first key exchange occurs over local TCP before encryption is established;
- the client trusts the discovered local Roof endpoint;
- stronger local pairing or out-of-band verification may be required later.

10.5 Registered Client Authentication

The intended registered-client authentication model is stronger than decrypt-only validation.

Successful AES payload decryption should not be the only proof of client validity. Registered clients should also complete a nonce challenge process before being fully trusted for command execution.

The intended model is:

1. client communicates using stored AES credentials;
2. Engine generates or validates a nonce challenge;
3. client proves possession of the expected credentials and client identity;
4. Engine marks the client as verified only after successful challenge completion.

Required registered-client checks should include:

- successful AES decryption;
- valid JSON request structure;
- registered client identity match;
- nonce challenge verification;
- command-level validation.

Current implementation status:

- nonce challenge verification is accepted as the intended security direction;
- nonce challenge enforcement is currently disabled until further implementation and product decisions are made;
- the current implementation may continue using decrypt-and-validate behaviour temporarily;
- decrypt-only validation must be documented as a transitional state rather than the final security mode
- The decrypted packet identity must match the registered client whose key decrypted it and any identity already bound to the connection.

10.6 Key Management

Roof Engine stores configuration encryption material and per-client network encryption credentials in protected binary files under /etc/Roof/keys.

The configuration key file is: /etc/Roof/keys/roof_config.key
A registered client's network key file is: /etc/Roof/keys/{clientId}.key

Each file contains a 32-byte AES-256 key followed by a 16-byte IV. Client network credentials are loaded from the client-specific key file when required and are not stored in the current RoofClientConfig object.

Supported key-related commands include:

- ROTATE_ENCRYPTION;
- ROTATE_ROOTPASSWORD;
- ROTATE_CLIENTPASSWORD.

Manual encryption rotation is supported through command flow.

Automatic credential rotation is accepted as a future security requirement, but it is currently disabled until further decision.

Manual rotation persists replacement credentials before announcing success. The response is encrypted using the old cached credentials, after which the current client's cache is updated.

The Engine should eventually be able to rotate client encryption credentials based on defined triggers, such as:

- access count;
- time interval;
- manual administrative action;
- security event;
- suspected credential compromise.

Before automatic rotation is enabled, the following must be defined:

- rotation trigger conditions;

- client acknowledgement requirements;
- recovery behaviour if rotation fails;
- whether old credentials remain temporarily valid;
- how credential divergence between client and Engine is resolved;
- how rotation events are logged;
- how rotation is surfaced to RoofDeck or other clients.

Current implementation status:

- manual encryption rotation is supported;
- automatic credential rotation is currently disabled;
- access-count-based rotation may be logged or detected;
- access-count-based rotation must not be treated as active automatic enforcement until enabled by further decision.

10.7 OS-Level Isolation

Roof Engine maps clients to Linux users.

Isolation principles:

- one Linux user per client;
- one shared playground accessible to accounts in roofsftpusers
- SFTP jail per client user/group;
- root-owned home directory;
- restricted writable playground directory;
- file ownership and permission enforcement after creation.

10.8 SFTP Jail Model

CREATE_ROOF_USER configures client-level SFTP access.

The process:

1. create Linux user;
2. set password;
3. attach username to registered client config;
4. create or use roofsftpusers group;
5. add user to group;
6. append SFTP jail block to /etc/ssh/sshd_config;
7. Roof SFTP accounts use the shared chroot /srv/roof and start in /playground through internal-sftp -d /playground -u 0007.
8. restart SSH.

Current implementation status:

- SFTP / Linux user setup is intended to be idempotent;
- the roofsftpusers group must not cause failure if it already exists;
- user creation must check whether the user already exists before running adduser;
- SFTP jail blocks must not be appended repeatedly to sshd_config;
- final ownership and permission state must be validated after setup;
- SSH should only be restarted after configuration validation succeeds.

Full idempotent provisioning remains incomplete. Existing requested usernames are rejected, and the examined SSH restart paths do not run sshd -t first.

10.9 Shell Command Trust Boundary

Roof Engine executes several privileged system operations, including:

- Linux user creation;
- password setting;
- file ownership changes;
- file permission changes;
- SSH/SFTP configuration;
- Wi-Fi management;
- systemd service setup;
- binary replacement.

These operations cross privileged OS boundaries and must be treated as high-risk.
Shell command hardening is mandatory.

Required behaviour:

- avoid direct shell string interpolation where possible;
- prefer argument-safe process execution;
- validate all client-derived inputs before OS execution;
- canonicalise and constrain filesystem paths;
- reject unsafe usernames, SSIDs, passwords, and paths where appropriate;
- avoid exposing secrets in logs;
- distinguish recoverable command failure from security-sensitive failure;
- restrict privileged operations to explicitly approved command paths;
- treat shell execution as a high-risk trust boundary.

Current implementation status:

- shell command execution has been hardened through shared safe execution helpers;
- usernames are validated before being passed to OS commands;
- filesystem paths are validated and canonicalised before privileged operations;
- privileged operations are restricted to explicit approved command paths;
- command handlers must avoid exposing secrets in logs;
- remaining shell-facing operations should continue to be reviewed as high-risk trust-boundary code.

11. Client Connection Flows

11.1 Local First-Time Registration Flow

Local registration flow:
Client
|
| TCP connect to port 8671

v
ClientManagerService
|
| Add unverified RoofClient
v
ClientManager
|
| TCP INFO REQUEST
v
Engine generates AES key and IV
|
| TCP_ENCRYPTION_INFO
v
Client sends encrypted REGISTER REQUEST
|
v
Engine verifies registration conditions
|
v
Engine stores client credentials
|
v
Engine returns encrypted registration success

Registration is allowed when:

- initial setup is incomplete;
- no registered clients exist;
- request is plain JSON;
- first request token is TCP INFO REQUEST;
- follow-up encrypted request begins with REGISTER REQUEST.

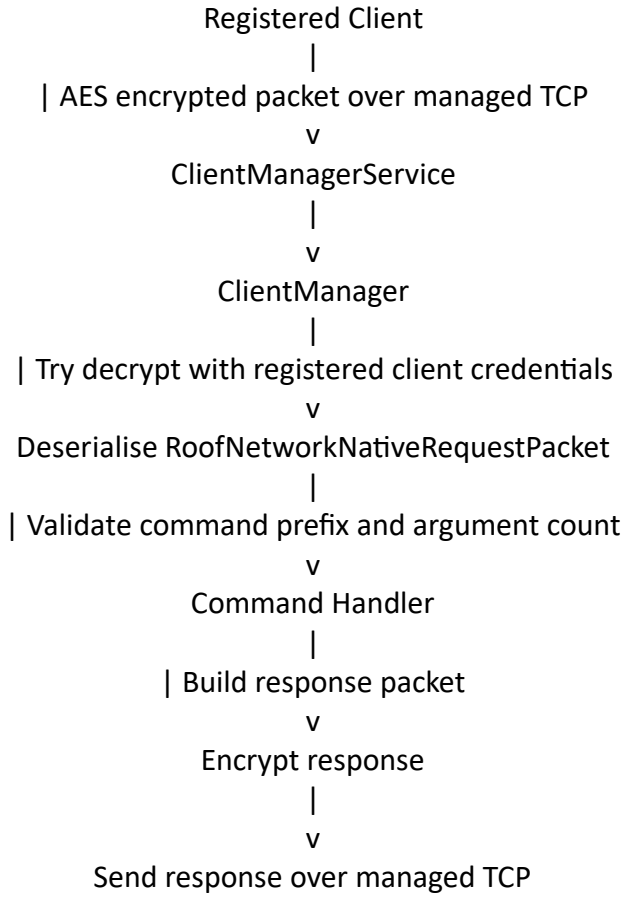
After successful first-time registration and setup finalisation:

- HasCompletedInitialSetup is marked complete;
- registration finalisation no longer directly owns Setup AP shutdown inside the registration handler;
- Setup AP shutdown is managed by Setup AP lifecycle logic and runtime heuristics;
- Setup AP may remain active temporarily where recovery heuristics still require local access, such as when internet connectivity remains unavailable;
- Setup AP cleanup must occur through the Setup AP lifecycle manager rather than being tightly coupled to the client registration handler.

Registration requests must use a 32-character hexadecimal client ID. The encrypted registration follow-up must match the identity recorded during the initial TCP information exchange.

11.2 Registered Local Command Flow

Registered local command flow:



Command dispatch awaits handler completion and serialises commands for each client object.

11.3 Remote PathFinder Command Flow

Remote command flow:



Command Handler
|
| Encrypted response wrapped for PathFinder
v
PathFinder
|
v
Remote Client

Remote command acceptance requires:

- sender identifier exists in registered clients;
- payload is not PathFinder heartbeat;
- payload decrypts using registered credentials;
- packet deserialises into a valid command request.

Remote registration is explicitly forbidden.

REGISTER REQUEST and TCP INFO REQUEST must only be accepted over the local managed TCP channel. Registration commands received from remote-mode clients or PathFinder-routed packets must be rejected or ignored with a clear logged reason.

11.4 Local-Only Multi-Client Registration

Roof Engine must support registration of additional clients after initial setup, but only through local network mode.

Remote registration through PathFinder must not be supported.
Additional client registration must require local network presence. Unknown remote clients must not be able to self-register through the relay path.

Required behaviour:

- allow additional client registration through local network onboarding;
- reject or ignore remote registration attempts through PathFinder;
- distinguish first-time setup registration from additional client registration;
- require local network presence for onboarding;
- ensure unknown remote clients cannot register through PathFinder;
- define an explicit local pairing or local onboarding process.

Current implementation status:

- multi-client registration is required by specification;
- registration must remain local-network-only;
- remote PathFinder-routed registration must be rejected;
- registration-specific handlers must guard against remote-mode registration attempts;
- rejection should produce a clear logged reason or structured failure response where appropriate.

Implementation note:

Additional local-client registration is implemented after setup completion. Remote registration remains rejected. Additional registration preserves the existing setup-completion state.

12. Networking Interface

12.1 Local Discovery

Local presence discovery uses UDP.

Setting	Value
UDP port	6969
Request payload	ROOF_PRESENCE_BEAM
Response payload	ROOF_PRESENCE_BEAM_ALIVE

The Engine listens for presence beam requests and replies with the alive response.

On macOS, local-device discovery must not perform active ICMP subnet ping sweeps. Instead, discovery should read ARP table candidates, filter unusable addresses, de-duplicate results, and return usable IPv4 candidates only.

Usable IPv4 candidates must exclude:

- non-IPv4 values;
- 0.0.0.0;
- loopback 127/8;
- link-local 169.254/16;
- multicast or reserved 224.0.0.0/4 and above.

The macOS ARP command should use numeric output, such as arp -an, to avoid DNS/name resolution overhead.

Windows behaviour may continue combining subnet ping scanning with ARP table candidates.

12.2 Setup AP Networking

Roof Engine provides a temporary Setup AP for local onboarding and recovery access when internet connectivity is unavailable. The AP is managed through NetworkManager and uses a dedicated wireless interface.

Configuration

Setting	Current value
NetworkManager connection name	RoofAP
SSID	Roof Setup

Base wireless interface	wlan0
AP interface	ap0
Wireless mode	ap
Wireless band	bg
IPv4 method	shared
Connection autoconnect	disabled
Wireless security	Open network

The current implementation assumes the interface names wlan0 and ap0. They are not supplied through runtime configuration.

Startup Behaviour
In IA, Beta and Release builds, Engine checks internet availability after configuration initialisation. If internet connectivity is unavailable, Engine attempts to create and start the Setup AP.

HasCompletedInitialSetup affects the startup diagnostic message but does not independently determine whether the AP starts. Consequently:

- An offline device may start the AP even after initial setup is complete.
- Incomplete setup does not, by itself, start the AP when internet connectivity is available.
- Completing initial setup does not directly stop the AP.

Debug startup skips this initial AP creation path. However, the runtime AP monitor described below also operates in Debug builds.

AP creation or activation failure is logged. Such failure does not, by itself, prevent Engine from continuing with direct local-network setup.

Runtime Monitoring
ClientManagerService evaluates the Setup AP requirement every 15 seconds.

Each evaluation checks whether nmcli is available. If it is unavailable, Engine reports the monitoring limitation and skips AP management for that cycle. Later cycles check again, allowing monitoring to resume if nmcli becomes available.

When the required tooling is available:

- If the AP is active and internet connectivity is available, Engine attempts to stop it.
- If the AP is inactive and internet connectivity is unavailable, Engine attempts to start it.
- Otherwise, the existing AP state is retained.

Internet availability is determined by the Engine's ICMP reachability check against 8.8.8.8. AP lifecycle decisions therefore depend on that check rather than a comprehensive assessment of internet access.

Each monitoring cycle catches and logs its own failures. ClientManagerService cancels the monitoring loop during termination.

Profile Creation and Activation
Setup AP lifecycle operations are serialised through a shared semaphore.

Creation checks wireless readiness, ensures ap0 exists and creates the RoofAP connection profile. When recreating the profile, Engine removes an existing RoofAP profile before applying the expected configuration.

Activation performs the following steps:

1. Check that wlan0 exists, the Wi-Fi radio is enabled, rfkill reports Wi-Fi as unblocked and wlan0 is up.
2. Ensure ap0 exists, creating it from wlan0 where supported.
3. Bring ap0 online.
4. Read the RoofAP profile and compare its connection type, interface, SSID, wireless mode, band, IPv4 method and autoconnect setting with the expected values.
5. Recreate the profile if it is missing or invalid.
6. Activate RoofAP on ap0.
7. Verify that the expected connection is active.

These checks also apply when activation is requested by runtime monitoring. The periodic path can therefore recreate a missing or invalid profile; profile creation is not restricted to startup.

If the AP is already active, activation returns successfully without starting a duplicate connection.

Wireless Recovery
If creation or activation fails, Engine may attempt wireless recovery when wlan0 exists and the recovery guard permits another attempt.

Recovery can:

- unblock Wi-Fi through rfkill;
- restart NetworkManager when the wireless state indicates that a restart is needed;
- enable the Wi-Fi radio;
- bring wlan0 online;
- remove an existing ap0 interface;
- recreate and bring ap0 online; and
- recreate the RoofAP profile.

The activation path retries AP activation after successful recovery.

A failed disruptive recovery sets a flag that suppresses subsequent recovery attempts. Successful AP activation, or detection of an already-active AP, clears that flag. Recovery is therefore not permanently limited to one attempt per process.

Normal activation continues to check and ensure ap0 independently of this recovery guard. Creating ap0 has no permanent once-per-runtime restriction.

Shutdown and Security Boundaries
Stopping the Setup AP first checks whether it is active, requests that NetworkManager bring the connection down, and verifies that it is no longer active.

The AP is an open network. Its availability is currently constrained by connectivity-based lifecycle management and disabled NetworkManager autoconnect.

Disabled autoconnect prevents NetworkManager from automatically activating the profile. It does not prevent Engine from explicitly starting the AP during offline startup or runtime recovery.

Setup AP operations are internal Engine lifecycle operations and are not exposed as ordinary remote-capable Wi-Fi commands. Their privileged networking effects and open-network exposure remain security and reliability considerations.

12.3 Local Managed TCP Channel

Local command traffic uses managed TCP.

Setting	Value
Listen address	0.0.0.0
Port	8671
Frame format	4-byte big-endian length prefix + payload
Data prefix	MTCP_DATA:
Heartbeat payload	MTCP_HEARTBEAT
Heartbeat recurrence	5 seconds
Timeout	15 seconds
Max frame size	64 MiB

Managed TCP emits data payloads to Engine after stripping the MTCP_DATA: prefix.

12.4 Remote PathFinder WebSocket Channel

Roof Engine reads its PathFinder destination from the PATHFINDER_URL constant in PathFinderService. The current source assigns the non-URL placeholder PATHFINDER_PUBLIC_URL.

A valid WebSocket endpoint must be supplied in source before remote connectivity can operate. This value is not currently loaded from runtime configuration.

The standalone PathFinder relay listens on TCP port 8071; public endpoint routing and TLS termination are deployment responsibilities.

Remote packets use RoofWebSocketPacket.

Fields:

Field	Meaning
SenderIdentifier	Logical sender identifier.
ForwardingToIdentifier	Logical destination identifier used for relay routing.
SenderFingerprint	Relay-assigned sender connection fingerprint.
Payload	Encrypted command payload or heartbeat marker.

PathFinder heartbeat behaviour:

- Engine sends WS_HEARTBEAT to the fixed PathFinder identifier;
- recurrence is 5 seconds;
- timeout is 15 seconds.

PathFinder-routed client heartbeat behaviour:

- Engine forwards PF_HEARTBEAT;
- recurrence is 5 seconds;
- timeout is 15 seconds.

Reconnect behaviour:

- uses a fixed 20 second reconnect delay;
- includes up to 20 percent jitter;
- cleans up failed provider initialisation attempts before retrying;
- publishes the provider to ClientManager only after successful initialisation;
- waits up to 10 seconds for the connection to become active after initialisation;
- clears the provider if the connection does not become active within the grace window.

Ordinary PathFinder connection failures should be logged as warnings using the exception type rather than full noisy exception output. Full exception logging should be reserved for unexpected or diagnostic cases where detailed stack traces are required.

Engine rejects remote payload text exceeding 16 MiB of UTF-8 data before client-state creation or decryption.

12.5 Managed UDP Availability

Managed UDP is available through Roof.Common but is not the primary Engine command path.

Managed UDP supports:

- sequence numbers;
- ACK packets;
- retry;
- heartbeat;
- timeout.

Current Engine command traffic primarily uses:

- UDP only for local presence discovery;
- managed TCP for local commands;
- WebSocket for remote PathFinder commands.

12.6 Heartbeat & Disconnection Model

Default heartbeat settings across providers:

Concern	Value
Heartbeat recurrence	5 seconds
Disconnection timeout	15 seconds

Heartbeat is used to detect stale TCP, UDP, WebSocket, and PathFinder-routed connections.

Heartbeat loops must include explicit sleeps when no provider is available, when PathFinder is disconnected, and after each scan of connected clients. Current expected delay is 500ms for idle or post-scan heartbeat loop waits. This prevents heartbeat processing from becoming a tight CPU-hogging loop.

WebSocketProvider receive behaviour must treat the following as disconnection events:

- socket state is not open;
- a close frame is received;
- a WebSocketException occurs;
- the listener exits due to a disconnected socket.

JSON deserialisation failures should not terminate the WebSocket listener. A malformed packet should be ignored or skipped so that one invalid packet does not kill the connection loop.

A client heartbeat refreshes only an existing verified remote session with a matching fingerprint. Heartbeats do not create sessions.

New remote sessions receive an initial heartbeat timestamp, allowing timeout even when the first heartbeat never arrives.

12.7 Trust Model

Local connections are trusted only after they satisfy the registration or registered-client command requirements.

Remote connections are trusted only if:

- PathFinder sender identifier is registered locally;
- encrypted payload decrypts correctly;
- payload parses into a valid request packet.

PathFinder itself is not trusted with plaintext application payloads.

13. API/Interface Definitions

13.1 Native Request Packet

RoofNetworkNativeRequestPacket:

Field	Type	Meaning
ClientID	string	Registered client identifier.
RoofIP	string	Roof IP or routing target depending on context.
Requests	List<string>	Command tokens. First item is command prefix. Remaining items are arguments.

Canonical shape:

```
{
  "ClientID": "client-id",
  "RoofIP": "optional-roof-ip",
  "Requests": ["COMMAND", "arg1", "arg2"]
}
```

After registration, this JSON is AES-encrypted before transport.

13.2 Native Response Packet

RoofNetworkNativeResponsePacket:

Field	Type	Meaning
Status	bool	Success or failure.
Responses	List<string>	Response tokens and payloads.

Canonical shape:

```
{
  "Status": true,
  "Responses": ["RESULT_PREFIX", "payload"]
}
```

AddResponse():

- lazily initialises the response list;
- suppresses duplicate response strings.

13.3 Command Dispatch Semantics

Command dispatch uses a prefix-based model.
The dispatch table maps command definitions to handlers. The first item is the command prefix. Remaining items represent expected arguments.

Dispatch behaviour:

1. Read Requests[0].
2. Match command prefix case-insensitively.
3. Validate argument count exactly.
4. Invoke handler with client and arguments.
5. Handler builds and sends response.

Current caveat:

- handler tasks are invoked through DynamicInvoke;
- returned handler tasks are awaited;
- handler exceptions are caught and logged during awaited dispatch.

13.4 Response Payload Pattern

Common response layout:

Index	Meaning
Responses[0]	Result prefix.
Responses[1]	Main payload, ID, hash, data, credential, or error.
Responses[2+]	Additional payload or error details.

Examples:

```
1. ["CONNECT_COMMAND_RESULT"]
2. ["GET_ROOFID_COMMAND_RESULT", "{roofIdentifier}"]
3. ["GET_FILE_DATA_COMMAND_RESULT", "{hashedPath}", "{base64Content}"]
4. ["ROTATE_ENCRYPTION_COMMAND_RESULT", "{newKey}", "{newIV}"]
```

14. Command Catalogue

14.1 Client Management Commands

Command	Arguments	Purpose	Result Prefix
CONNECT	none	Marks client verified, initialises FS manager, increments access count.	CONNECT_COMMAND_RESULT
ROTATE_ROOTPASSWORD	none	Generates and sets Linux roof password.	ROTATE_ROOTPASSWORD_COMMAND_RESULT
ROTATE_CLIENTPASSWORD	none	Generates and sets Linux password for client user.	ROTATE_CLIENTPASSWORD_COMMAND_RESULT
ROTATE_ENCRYPTION	none	Generates replacement AES credentials, persists them to the client-specific key file, sends the response using the old cached credentials, and then updates the current client's cached key and IV.	ROTATE_ENCRYPTION_COMMAND_RESULT
CREATE_ROOF_USER	username, password	Creates Linux user, configures SFTP jail, creates playground.	CREATE_ROOF_USER_COMMAND_RESULT
GET_ROOFID	none	Returns Roof device identifier.	GET_ROOFID_COMMAND_RESULT
RESTART_ROOF	None	Runs operating-system synchronisation, waits five seconds, sends a successful response and requests a reboot through ShutdownManager. Coordinated shutdown is attempted before the operating-system reboot command.	RESTART_ROOF_COMMAND_RESULT

RESTART_ROOF is currently available through the normal registered-client command dispatch path. The handler does not reject remote-mode clients. Consequently, a registered PathFinder-routed client can currently request a device restart.

The handler first runs sudo sync. If the command reports an error, Engine sends a failed RESTART_ROOF_COMMAND_RESULT response and does not request reboot.

Otherwise, it waits five seconds, sends a successful response and calls ShutdownManager.RequestReboot(). ShutdownManager attempts service termination, completion of tracked client work and configuration saving before invoking sudo reboot.

14.2 Wi-Fi Commands

Command	Arguments	Purpose	Result Prefix
GET_WIFI_LIST	none	Lists locally visible Wi-Fi SSIDs.	GET_WIFI_LIST_COMMAND_RESULT
CONNECT_TO_WIFI	ssid, password	Connects device to Wi-Fi network.	CONNECT_TO_WIFI_COMMAND_RESULT
GET_CURRENT_WIFI	none	Returns current Wi-Fi SSID.	GET_CURRENT_WIFI_COMMAND_RESULT
FORGET_CURRENT_WIFI	None	Deletes the NetworkManager connection profile currently active on wlan0.	FORGET_CURRENT_WIFI_COMMAND_RESULT

Wi-Fi commands, including FORGET_CURRENT_WIFI, are rejected in remote mode.

Setup AP lifecycle operations remain internal Engine operations and are not exposed as normal remote-capable Wi-Fi commands.

Setup AP operations must not be exposed to remote PathFinder-routed clients.

14.3 File-System Commands

Command	Arguments	Purpose	Result Prefix
GET_BASE_DIRECTORY	none	Returns base directory metadata.	GET_BASE_DIRECTORY_COMMAND_RESULT
GET_DIRECTORY	hashedPath	Returns directory metadata.	GET_DIRECTORY_COMMAND_RESULT
GET_DIRECTORIES	hashedPath, serialised_json	Its behaviour remains unsupported and returns Status=false with Invalid operation.	GET_DIRECTORIES_COMMAND_RESULT

Command	Arguments	Purpose	Result Prefix
CREATE_DIRECTORY	hashedPath, serialised_json	Creates physical and metadata directory.	CREATE_DIRECTORY_COMMAND_RESULT
UPDATE_DIRECTORY	hashedPath, serialised_json	Updates/moves directory.	UPDATE_DIRECTORY_COMMAND_RESULT
COPY_DIRECTORY	copyingHashedPath, targetHashedPath	Copies directory under target.	COPY_DIRECTORY_COMMAND_RESULT
DELETE_DIRECTORY	hashedPath	Deletes non-base directory.	DELETE_DIRECTORY_COMMAND_RESULT
GET_FILE	hashedPath	Returns file metadata.	GET_FILE_COMMAND_RESULT
CREATE_FILE	hashedPath, serialised_json	Creates empty file and metadata.	CREATE_FILE_COMMAND_RESULT
UPDATE_FILE	hashedPath, serialised_json	Writes, appends, or updates file metadata.	UPDATE_FILE_COMMAND_RESULT
DELETE_FILE	hashedPath	Deletes file.	DELETE_FILE_COMMAND_RESULT
GET_FILE_DATA	hashedPath	Returns Base64 file content.	GET_FILE_DATA_COMMAND_RESULT

15. Wi-Fi Management

15.1 Wi-Fi Scope

Wi-Fi management is a local-only Engine capability. Remote clients must not execute Wi-Fi configuration commands through PathFinder. This prevents remote clients from accidentally or maliciously disconnecting the Roof device from its active network path.

15.2 Setup AP Mode

Roof Engine supports Setup AP mode as a temporary local access network for initial onboarding and recovery.

The implemented AP lifecycle is driven by internet availability.

Startup AP creation is attempted in IA, Beta and Release builds when internet connectivity is unavailable.
Runtime monitoring attempts activation while offline and shutdown when connectivity returns. Initial setup completion is not an independent start or stop condition.

The setup AP must:

- be created only when setup or recovery conditions require it;
- use NetworkManager AP mode;
- use `802-11-wireless.mode ap`;
- use the dedicated AP interface `ap0`;
- use `wlan0` as the base wireless interface for AP-interface recovery;
- use wireless band `bg`;
- use shared IPv4 networking;
- have autoconnect disabled;
- be started only when not already active;
- be stopped through lifecycle cleanup when no longer required;
- be deleted and recreated if an invalid or partial configuration exists;
- fail safely without preventing direct local setup;
- avoid exposing credentials or setup-sensitive values in logs.

The setup AP is currently configured as an open network.

Because the Setup AP is open, its lifecycle must be tightly constrained. It must not autoconnect on boot and should be stopped once setup or recovery access is no longer required.

15.3 Wi-Fi Listing

GET_WIFI_LIST performs:

1. sudo nmcli device wifi rescan;
2. nmcli -t -f IN-USE,BSSID,SSID,CHAN,SIGNAL,SECURITY device wifi list;
3. parses terse output;
4. groups APs by SSID;
5. ranks APs;
6. returns SSIDs.

Ranking prefers:

- stronger signal;
- 5 GHz networks;
- active networks;
- secured networks.

15.4 Wi-Fi Connection

CONNECT_TO_WIFI:

1. rejects remote clients;

2. validates that SSID and password arguments are present before indexing request arguments;
3. copies SSID and password into named local variables for clearer flow;
4. enables Wi-Fi radio using NetworkManager;
5. ensures wlan0 is managed by NetworkManager;
6. waits briefly for the wireless stack to settle;
7. rescans available Wi-Fi networks;
8. treats Wi-Fi rescan as failed only when nmcli output contains Error:;
9. verifies the target SSID exists;
10. deletes any stale Roof-created Wi-Fi profile for the target network;
11. creates a Roof-owned NetworkManager profile named Roof-WiFi-{ssid};
12. configures the profile with WPA-PSK settings;
13. brings the created profile up on wlan0;
14. returns a structured success or failure response.

The handler should explicitly create and activate a Roof-owned NetworkManager connection profile rather than relying on nmcli device wifi connect.

The connection profile name must use the pattern: Roof-WiFi-{ssid}
This avoids colliding with stale user-created or system-created NetworkManager profiles that happen to use the SSID as the profile name.

SSID, password, and profile values must be shell-quoted before being passed to shell commands. The current implementation uses a ShellQuote() helper to safely single-quote these values.

The Wi-Fi connection handler should return detailed failure responses for:

- invalid Wi-Fi connection request;
- Wi-Fi rescan failure;
- profile creation failure;
- connection activation failure;
- target SSID not found.

The success response should include: Wifi connected successfully.

15.5 Current Wi-Fi Detection

GET_CURRENT_WIFI runs a command equivalent to: nmcli -t -f active,ssid dev wifi | grep '^yes' | cut -d: -f2

If no active Wi-Fi is found, Engine returns: Not connected to a wifi.

15.6 Forget Current Wi-Fi

FORGET_CURRENT_WIFI removes the NetworkManager connection profile currently active on wlan0.
Current behaviour:

- rejects remote-mode clients;

- runs nmcli -g GENERAL.CONNECTION device show wlan0 to identify the active connection profile;
- returns Not connected to a wifi. when no active connection profile is found;
- shell-quotes the returned NetworkManager profile name;
- runs sudo nmcli connection delete {connectionProfile};
- treats an empty deletion result, an Error: result, or a result that does not contain successfully deleted. as failure;
- returns FORGET_CURRENT_WIFI_COMMAND_RESULT as the response prefix;
- returns a successful response without an additional payload when deletion succeeds.

This command deletes only the currently active NetworkManager connection profile. Engine does not currently provide general saved-network enumeration, bulk deletion, or selection of an arbitrary saved profile.

15.7 Wi-Fi Limitations

Current limitations:

- local-only;
- depends on nmcli;
- assumes NetworkManager-managed Wi-Fi;
- Setup AP depends on wlan0 availability, ap0 creation, Wi-Fi hardware support, and successful WLAN recovery;
- Setup AP is open during setup/recovery access and must therefore be strictly lifecycle-constrained;
- Failed disruptive wireless recovery suppresses further recovery attempts until successful activation or already-active AP handling clears the recovery flag.
- AP-interface existence and activation are checked during normal AP activation. Creation of ap0 is not permanently limited to one attempt per runtime.
- no structured Wi-Fi error taxonomy;
- no general saved-network enumeration, arbitrary profile selection, or bulk saved-network management; the current command surface can only delete the connection profile active on wlan0;
- setup AP lifecycle is internal setup infrastructure, not a normal remote-capable Wi-Fi command surface.
- Wi-Fi connection reliability depends on NetworkManager correctly managing wlan0. Before connecting, Engine now explicitly enables Wi-Fi radio, marks wlan0 as managed, waits for the wireless stack to settle, deletes stale Roof-owned profiles, creates a new Roof-owned profile, and activates that profile.
- Wi-Fi command hardening is still required because SSID and password values cross shell-command boundaries. Shell quoting has been added for SSID, password, and profile values, but Wi-Fi command handling should continue to be reviewed as privileged OS-boundary code.
- forgetting the active Wi-Fi profile immediately changes the device's network state and may remove its current connectivity path;
- FORGET_CURRENT_WIFI depends on NetworkManager returning a valid active connection-profile name and deletion output containing the expected success text.

16. Update System

16.1 Release Channels

Build configurations include:

- Debug;
- Release;
- IA;
- Beta.

Assembly informational versions currently follow channel-specific values, such as:

Channel	Current Informational Version
Debug	0.0.0.0-debug
IA	2.3.0.0-ia
Beta	2.2.0.0-beta
Release	0.0.0.0-release

16.2 Metadata Sources

The release server base address is defined by RELEASE_CHANNEL_BASE_URL in ReleaseServerIngressManager.

The current source assigns the placeholder RELEASE_CHANNEL_BASE_URL rather than a usable server URL.

A valid base address must be supplied in source before metadata retrieval and artifact downloads can operate. This address is not currently loaded from runtime configuration.

Metadata paths:

Compile Flag	Metadata Path
IA or DEBUG	/ia/artifact_versions.json
BETA	/beta/artifact_versions.json
RELEASE	/pa/artifact_versions.json

Expected metadata shape:

```
{
  "artifacts": [
    {
      "RoofEngine": {
        "Version": "...",
        "LINUX_ARM64_Hash": "..."
      }
    }
  ]
```

```
10. }
```

16.3 Update Check Algorithm

Current update check algorithm:

1. If current process name is not Roof.Engine, finalise replacement.
2. Fetch latest version metadata.
3. Fetch official Linux ARM64 Engine hash.
4. Compute SHA-256 of the current running executable.
5. Compare local version and hash against official metadata.
6. If both match, no update is required.
7. If version or hash mismatch, download new binary.
8. Download the new binary to a staging path.
9. Compute SHA-256 of the downloaded artifact.
10. Compare the downloaded artifact hash against official metadata.
11. Delete the downloaded artifact if the hash does not match.
12. Log hash mismatch as a critical update integrity failure.
13. Replace or restart only if the downloaded artifact hash matches official metadata.

UpdateService now uses a Task with PeriodicTimer rather than a manual Thread and Stopwatch loop. The update loop waits for a 60 second timer tick and supports cancellation through CancellationTokenSource.

16.4 Binary Replacement

Binary replacement is Linux-oriented.

The replacement flow may:

- set execute permissions;
- move new binary over the current binary;
- execute or restart the new binary;
- exit the old process.

16.5 Restart Behaviour

Debug behaviour:

- uses direct replace-and-exec flow;
- sets chmod 755;
- moves binary over target;
- uses execv to replace current process.

Release/Beta/IA behaviour:

- starts shell command;
- sleeps briefly;
- applies permissions;
- moves new binary;
- starts target with sudo -n;
- exits current process

16.6 Update Integrity Requirements

The required update integrity model is:

1. fetch metadata;
2. compare version;
3. download new binary to a staging path;
4. compute SHA-256 of downloaded file;
5. compare downloaded file hash with official hash;
6. replace binary only when hash matches;
7. discard binary on mismatch;
8. log critical error on mismatch;
9. retain current working binary when validation fails.

Current implementation status:

- downloaded update artifact hash verification is implemented;
- invalid downloaded artifacts must be deleted;
- active binary replacement must only occur after downloaded artifact verification succeeds.

16.7 Update Limitations

Current limitations:

- Linux-only update flow;
- Linux ARM64 artifact assumption;
- update polling currently uses a 60 second PeriodicTimer after setup, which may still be too frequent for production release-server load, but no longer uses a tight manual timing loop.
- update restart uses privileged system operations and must remain within the hardened command-execution boundary.

17. Deployment & Runtime Model

17.1 Target Hardware

Primary hardware target:

- Raspberry Pi 5;
- ARM Cortex-A76;
- ARM64 runtime;
- GB–16 GB RAM range;
- Wi-Fi;
- Ethernet;
- USB;
- PCIe / expandable storage.

17.2 Operating System

Primary OS target: Raspberry Pi OS Lite X64 / Debian 12-compatible Linux
Engine assumes a minimal Linux runtime environment.

17.3 Runtime Environment

Runtime:

- .NET 10;
- NativeAOT enabled;
- Linux ARM64 optimised;
- invariant globalisation enabled;
- swap-supported memory scaling where required.

17.4 Host Dependencies

Runtime dependencies include:

- /bin/bash;
- systemctl;
- dpkg;
- apt-get;
- OpenSSH server;
- sudo;
- nmcli;
- adduser;
- chpasswd;
- usermod;
- groupadd;
- getent;
- Linux filesystem permission tools.
- zstd, used to compress rotated log files;
- permission to create, compress, inspect, and delete files in the Engine log directory.
- ACL utilities, including setfacl; Engine attempts to install the acl package when required.

Runtime dependencies also include:

- NetworkManager AP-mode support;
- wlan0 or equivalent supported wireless interface for setup AP mode;
- ability to create a dedicated `ap0` AP interface from `wlan0`;
- Wi-Fi hardware capable of AP mode;
- iw for AP-interface creation;
- ip` for interface state management;
- rfkill for Wi-Fi unblock recovery;
- nmcli support for creating, deleting, starting, stopping, and inspecting AP connections.

17.5 Startup Service

Startup unit path: /etc/systemd/system/roofenginestartup.service

A Debug startup-unit template exists in Program.cs. It expects /roof/Roof.Engine, uses /roof as its working directory, requests restart on failure and writes output to the journal.

Normal Debug startup does not call CreateStartupService(), so this template is not installed automatically through the current Debug startup path.

IA, Beta and Release unit behaviour:

- Type=oneshot;
- RemainAfterExit=yes;
- KillMode=none;
- detaches Engine using setsid -f;
- does not allow systemd to manage the detached child process lifetime directly.

This should be reviewed for production supervision, because a detached process may not benefit from standard systemd restart management.

17.6 Resource Manager Service Position

ResourceManagerService is active and registered through ServiceManager.

The service runs a cancellable PeriodicTimer loop every 60 seconds. Its currently implemented responsibility is log-file lifecycle management.

Current active responsibilities:

- check whether the current log has reached the 24-hour rotation interval;
- rotate Logger to a new timestamped .log file;
- compress the previous log using zstd -3 -q --rm;
- inspect compressed *.log.zst files;
- delete compressed log files whose last-write time is at least 30 days old.

The service currently logs that it is monitoring resource usage, but it does not implement CPU, memory, disk-capacity, thermal, network, process, GPU, I/O, service, or user-usage monitoring.

Existing SystemResourceManager methods for disabling Linux services remain present, but ResourceManagerService does not currently invoke them.

Future resource-management work must separately define:

- CPU and memory monitoring;
- disk-capacity and storage-pressure monitoring;
- thermal monitoring;
- service-health monitoring;
- resource thresholds;
- degraded-mode behavior;
- operation throttling or rejection;
- how resource state is surfaced to clients.

17.7 Platform Scope

Current platform support: Linux only

Windows and macOS are not supported for Engine runtime because Engine depends heavily on:

- Raspberry Pi hardware
- Linux users;
- Linux permissions;
- OpenSSH server;
- systemd;
- NetworkManager;
- Linux shell commands;
- Linux binary update flow.

18. Performance & Resource Constraints

18.1 Runtime Characteristics

Roof Engine uses:

- background service threads;
- async network callbacks;
- periodic heartbeat loops;
- recursive file-system indexing;
- update polling;
- on-demand file hashing;
- local network operations.
- Roof Engine also includes heuristic Setup AP lifecycle operations before initial setup completion or when internet connectivity is unavailable.

Recent performance work specifically reduced CPU usage from hot loops and tight polling. PathFinder heartbeat loops now sleep while disconnected or idle, WebSocket disconnection polling has been slowed, and UpdateService now uses PeriodicTimer rather than a manually calculated delay loop.

18.2 Network Constraints

Concern	Value
Managed TCP max frame	64 MiB
TCP heartbeat	5 seconds
TCP timeout	15 seconds
WebSocket heartbeat	5 seconds
WebSocket timeout	15 seconds

Concern	Value
Managed UDP retry interval	2 seconds
Managed UDP max retries	5

18.3 File-System Constraints

File hashing:

- uses a rented 128 KiB buffer;
- computes xxHash64 from file stream;
- preserves stream position where possible.

Indexing:

- recursive directory scan occurs during file-system manager construction;
- directory size calculation recursively walks child directories and files.

Risks:

- large client directories may slow startup or client connection;
- large files may stress memory if transferred as single Base64 payloads;
- recursive size calculation may become expensive.

18.4 Update Polling

UpdateService checks every: 60 seconds

Update checks occur only after initial setup is complete. UpdateService now uses PeriodicTimer and cancellation support rather than a manual thread and Stopwatch-based loop. The interval should still be reviewed before production to avoid unnecessary release server load.

18.5 Resource Risks

Current resource risks include:

- always-running background threads;
- recursive file indexing;
- large single-frame file transfers;
- Base64 expansion overhead;
- unmanaged task exceptions;
- heartbeat and polling loops must continue to be monitored for accidental regressions, although recent fixes added sleeps, blocking receive behaviour, slower polling, and PeriodicTimer usage to reduce CPU churn.
- ResourceManagerService is active, but its implemented scope is currently limited to log rotation, compression, and retention cleanup;
- the service runs at the highest thread priority even though its work is periodic and primarily I/O-oriented;
- log compression depends on the external zstd utility;

- failure to compress a rotated log leaves the old uncompressed log outside the current compressed-log retention scan;
- retention cleanup currently scans only *.log.zst files;
- CPU, memory, disk-capacity, thermal, and service-health monitoring remain unimplemented.
- malformed WebSocket packets should not kill listener loops;
- failed PathFinder provider initialisation must not leave stale active provider references;
- Wi-Fi connection attempts may still fail depending on NetworkManager state, stale profiles, or wireless stack readiness.

19. Logging & Observability

19.1 Logging Location

Logs are written through Roof.Common.Tools.Logger.
Log directory: {engine working directory}/Logs
Initial log filename: {process start time as yyyy-MM-ddTHHmmss}.log

Logger initially derives the filename from the process start time. ResourceManagerService subsequently rotates the active log after each 24-hour runtime interval.
A rotated log is compressed to: {timestamp}.log.zst

Logger uses a dedicated file lock to serialise file writes and log-file rotation.

19.2 Severity Levels

Supported log severity values:

- Neutral;
- Warning;
- Danger.

19.3 Logged Events

Roof Engine logs:

- version and compile flag;
- startup checks;
- internet availability;
- update check behaviour;
- service initialisation;
- PathFinder connection status;
- client connection and disconnection events;
- heartbeat timeout events;
- command warnings and failures;
- update failures;
- exception details;
- file-system operation failures.
- PathFinder provider initialisation failure;

- PathFinder inactive-after-initialisation grace timeout;
- PathFinder provider cleanup after failed initialisation;
- WebSocket non-open socket state detection;
- WebSocket close-frame disconnection;
- WebSocketException converted to disconnected event;
- Wi-Fi rescan failure;
- Wi-Fi profile creation failure;
- ·Wi-Fi connection activation failure;
- stale Roof Wi-Fi profile deletion;
- update timer cancellation;
- macOS ARP-table discovery behaviour.

Roof Engine also logs:

- Setup AP active/inactive checks;
- setup AP creation failure;
- setup AP startup retry failure;
- setup AP shutdown failure;
- WLAN unblock attempt;
- WLAN unblock skipped because it was already attempted this runtime;
- WLAN radio recovery failure;
- NetworkManager restart failure;
- ap0 interface creation failure;
- ap0 interface recovery skipped because it was already attempted this runtime;
- ap0 interface activation failure;
- Setup AP requirement monitor failure;
- Setup AP cleanup after internet connectivity is restored.
- setup AP lifecycle cleanup;
- configuration decryption failure;
- configuration integrity validation failure;
- update artifact hash mismatch;
- remote registration rejection.
- ResourceManagerService startup;
- periodic resource-manager execution;
- log rotation start and completion;
- rotated-log compression start and completion;
- log compression failure;
- compressed-log eviction;
- compressed-log eviction failure;
- Setup AP requirement evaluation;
- Setup AP start after internet loss;
- Setup AP stop after internet restoration;
- current-Wi-Fi forget requests and failures;
- Roof restart requests;
- operating-system synchronisation failure before restart;
- successful synchronisation before restart.

19.4 Current Observability Limitations

Current limitations:

- no structured telemetry;
- no metrics aggregation;
- no health endpoint;
- no dashboard;
- no central logging pipeline;
- no formal event IDs;
- no privacy-aware redaction policy fully defined;
- no performance counters.
- log rotation is time-based from the current process or last rotation and is not aligned to a fixed wall-clock boundary;
- log retention applies only to compressed *.log.zst files;
- no configured storage quota limits total uncompressed log growth after compression failures;
- log compression and eviction metrics are not exposed;
- resource monitoring is named and scaffolded but does not yet collect system-resource measurements.

19.5 Future Metrics

Future observability should include:

- active local clients;
- active remote clients;
- command count by type;
- failed command count;
- average command latency;
- PathFinder reconnect count;
- update check success/failure;
- file operation count;
- file operation failures;
- storage usage per client;
- memory usage;
- CPU usage;
- network bytes sent/received;
- heartbeat timeout count;
- config load/save failures.

20. Failure Modes & Recovery

20.1 Configuration Failure

Possible failures:

- missing config;
- malformed JSON;
- schema mismatch;

- hash mismatch;
- plaintext or corrupted credentials;
- failed migration.

Desired behaviour:
- create default config when missing;
- migrate older schemas safely;
- block startup on tamper detection;
- log critical failure;
- avoid running with invalid credentials.

Current implementation status:
- configuration encryption is implemented;
- configuration integrity validation is enforced on load;
- failed decryption, failed integrity validation, corrupt config, tampered config, unreadable config, and migration failure must produce a clear failure state;
- Engine must not silently continue with a fresh empty configuration after a critical config failure.

20.2 Network Failure

Local TCP/UDP failure behaviour:
- listener errors are logged;
- heartbeat timeout emits disconnection;
- malformed frames are ignored or rejected;
- oversized TCP frames are rejected.

Desired recovery:
- continue serving other clients;
- clean stale client state;
- avoid crashing Engine on malformed packets.

Setup AP failure behaviour:
- Setup AP creation failure is logged as a critical setup availability failure;
- Setup AP startup failure is logged as a critical setup availability failure;
- AP-interface unavailability triggers one-time recovery heuristics;
- WLAN recovery failure is logged as a critical wireless recovery failure;
- ap0 creation or activation failure is logged as a critical AP-interface recovery failure;
- direct-connection setup may continue even if Setup AP startup fails;
- Setup AP lifecycle failures must not crash the Engine startup sequence;
- if Setup AP is running due to internet unavailability, Engine should stop it after internet connectivity is restored.

PathFinder WebSocket failure behaviour:
- failed provider initialisation must detach handlers and dispose the provider;
- failed provider initialisation must not publish the provider as active;
- inactive providers must be cleared after the connection grace window expires;

- malformed WebSocket packets must not terminate the listener;
- WebSocket close frames and WebSocket exceptions must be converted into disconnection events;
- local operation must continue while PathFinder reconnects.

Wi-Fi connection failure behaviour:

- invalid SSID/password arguments must be rejected before argument indexing;
- Wi-Fi rescan failure must only be inferred from actual nmcli error output;
- stale Roof-created profiles should be removed before new profile creation;
- profile creation failure and activation failure must produce clear failure responses;
- Wi-Fi setup should avoid conflicting with stale NetworkManager profiles named directly after the SSID.

ClientManagerService checks internet connectivity and Setup AP state every 15 seconds. If internet connectivity is lost and the Setup AP is inactive, Engine attempts to start the existing RoofAP connection. If connectivity returns while the Setup AP is active, Engine attempts to stop it.

Setup AP activation validates the existing profile and recreates it when missing or invalid.

20.3 PathFinder Failure

PathFinder disconnect behaviour:

- Engine reconnects using a fixed 20 second delay;
- up to 20 percent jitter is added;
- failed provider initialisation is cleaned up before retrying;
- provider handlers are detached on failed initialisation;
- failed providers are disposed;
- ClientManager only receives the provider after successful initialisation;
- Engine waits up to 10 seconds for the provider to become actively connected;
- inactive providers are cleared after the grace period expires.

Remote clients become unavailable when PathFinder is disconnected. Local Engine operation must continue without PathFinder.

20.4 Update Failure

Update failure behaviour:

- failed update check logs warning/danger;
- invalid hash should abort update;
- existing binary should be retained;
- Engine should continue running where safe.

Required behaviour:

- verify downloaded binary hash explicitly;
- delete invalid downloaded artifacts;
- log downloaded artifact hash mismatch as a critical update integrity failure;

- avoid partial replacement;
- retain current working binary when validation fails;
- record last update status where supported.

20.5 File-System Failure

Possible failures:

- physical file missing;
- permission denied;
- invalid hashed path;
- invalid serialised DTO;
- failed chown/chmod;
- failed delete;
- failed directory move;
- path traversal attempt.

Desired behaviour:

- return structured failure response;
- log warning or danger;
- do not mutate cache if physical operation fails;
- keep parent-child metadata consistent.

20.6 SFTP/OS User Failure

Possible failures:

- adduser failure;
- group already exists;
- user already exists;
- chpasswd failure;
- SSH restart failure;
- jail config duplication;
- permission failure.

Required behaviour:

- make user creation idempotent;
- do not fail when roofsftpusers already exists;
- check whether a user already exists before running adduser;
- avoid duplicate SSH config blocks;
- validate final filesystem permissions;
- restart SSH only after configuration validation succeeds;
- distinguish fatal and non-fatal shell errors.

20.7 Command Failure

Command failure should return a structured response packet where possible.

Common failure strings include:

- Invalid argument.

- Internal engine exception occurred while handling the operation.
- Failed to deserialize directory.
- Failed to deserialize file.
- Failed to create physical directory.
- Failed to create physical file.
- Failed to find a local directory that matches the requested directory.
- Failed to find a local file that matches the requested file.
- Requested file does not exist.
- Directory does not exist.
- Cannot configure wifi remotely.
- Cannot connect to a wifi remotely.
- Wifi rescanning failed.
- Wifi not found.
- Wifi connection failed.
- Not connected to a wifi.
- Invalid wifi connection request.
- Wifi profile creation failed.
- Wifi connection failed.
- Wifi connected successfully.

FORGET_CURRENT_WIFI may fail when:

- the request arrives through a remote-mode client;
- wlan0 has no active NetworkManager connection;
- NetworkManager cannot identify the active profile;
- the profile deletion command fails;
- deletion output does not contain the expected success marker.

RESTART_ROOF may fail when:

- the operating-system sync command returns an error;
- service termination, tracked client work or configuration saving fails during coordinated shutdown;
- Engine lacks permission to execute sudo reboot; or
- the reboot command fails after the successful command response has already been sent.

RESTART_ROOF sends its success response before requesting coordinated shutdown and before executing the operating-system reboot command.

A successful response confirms completion of the handler's pre-reboot sequence; it does not confirm successful shutdown, configuration saving or operating-system restart.

21. Testing & Validation

21.1 Current State

No formal test suite is currently identified for Roof Engine and Roof.Common. Testing should be added before treating Engine as stable.

21.2 Required Unit Tests

Required unit test areas:

1. configuration creation;
2. configuration save;
3. configuration load;
4. config hash generation;
5. config migration;
6. AES encryption/decryption compatibility;
7. SHA-256 hashing;
8. xxHash64 path validation;
9. request packet serialisation;
10. response packet serialisation;
11. command argument validation;
12. file DTO serialisation;
13. directory DTO serialisation;
14. Wi-Fi AP parsing;
15. release metadata parsing;
16. update version comparison.
17. encrypted configuration save/load;
18. plaintext-to-encrypted configuration migration;
19. configuration integrity validation failure;
20. failed config decryption;
21. safe shell command validation;
22. username validation;
23. path canonicalisation;
24. setup AP lifecycle state checks;
25. Initial setup completion alone does not trigger AP startup or shutdown.
26. AP startup decisions follow internet availability and the applicable build configuration.
27. AP monitoring uses a 15-second interval.
28. downloaded update artifact hash verification;
29. remote registration rejection;
30. thread-safe client collection access.
31. Setup AP starts when internet is unavailable.
32. ClientManagerService Setup AP requirement monitor stops the AP after internet returns.
33. Setup AP uses ap0 rather than wlan0 for AP connection startup.
34. ap0 recovery is attempted when AP interface is unavailable.
35. Failed disruptive recovery suppresses subsequent recovery attempts.
36. Successful activation or already-active AP handling resets the recovery flag.
37. Normal activation ensures ap0 exists and is brought online.
38. Missing or invalid RoofAP profiles are recreated during activation.
39. Setup AP startup retries after WLAN and ap0 recovery.
40. Setup AP running check trims native command output before comparison.
41. PathFinder fixed reconnect delay and jitter calculation.
42. Failed PathFinder provider initialisation detaches handlers and disposes provider.

43. PathFinder provider is not published before successful initialisation.
44. Inactive PathFinder provider is cleared after grace timeout.
45. Heartbeat loop sleeps when disconnected or provider is unavailable.
46. Heartbeat loop sleeps after connected-client scan.
47. WebSocket listener treats non-open socket state as disconnected.
48. WebSocket listener handles close frames as disconnection events.
49. WebSocket listener continues after JSON deserialisation failure.
50. WebSocketException triggers disconnect handling.
51. WebSocket disconnection monitor uses 1000ms polling.
52. UpdateService uses PeriodicTimer tick behaviour.
53. UpdateService cancellation path stops the timer loop cleanly.
54. CONNECT_TO_WIFI rejects missing SSID/password arguments before indexing.
55. Wi-Fi rescan only fails on nmcli Error: output.
56. Wi-Fi profile name uses Roof-WiFi-{ssid}.
57. Stale Roof-created Wi-Fi profiles are deleted before profile creation.
58. Wi-Fi SSID, password, and profile values are shell-quoted.
59. macOS discovery returns ARP-table candidates without active subnet ping sweep.
60. IsUsableIPv4Address rejects loopback, link-local, zero, multicast, reserved, and non-IPv4 values.
61. configuration schema version 2;
62. RoofClientConfig excludes network key and IV fields;
63. client key-file binary layout;
64. invalid 48-byte configuration key-file length handling;
65. legacy client key and IV migration into /etc/Roof/keys/{clientId}.key;
66. FORGET_CURRENT_WIFI rejects remote-mode clients;
67. FORGET_CURRENT_WIFI handles no active Wi-Fi connection;
68. FORGET_CURRENT_WIFI shell-quotes the active connection-profile name;
69. FORGET_CURRENT_WIFI detects successful and failed profile deletion;
70. RESTART_ROOF stops when sync fails;
71. RESTART_ROOF sends its successful response before calling ShutdownManager.RequestReboot(), and ShutdownManager attempts service termination, completion of tracked client work and configuration saving before invoking sudo reboot.
72. Setup AP monitor starts the AP while internet is unavailable;
73. Setup AP monitor stops the AP after internet is restored;
74. Setup AP monitor cancellation terminates its PeriodicTimer loop;
75. ResourceManagerService cancellation terminates its PeriodicTimer loop;
76. log rotation occurs after 24 hours;
77. logger writes remain serialised during rotation;
78. rotated log compression uses the expected zstd arguments;
79. compression failure preserves and reports the uncompressed log;
80. compressed logs younger than 30 days are retained;
81. compressed logs at least 30 days old are evicted.Setup AP cleanup task stops AP after internet returns.

21.3 Required Integration Tests

Required integration tests:

1. local UDP presence response;
2. local managed TCP connection;
3. first-time registration handshake;
4. registered client CONNECT;
5. encrypted command dispatch;
6. local file-system CRUD;
7. SFTP user creation in controlled Linux test environment;
8. PathFinder reconnect;
9. PathFinder-routed command execution;
10. update metadata fetch;
11. binary hash validation;
12. systemd service generation.
13. In IA, Beta and Release builds, offline startup attempts Setup AP creation regardless of setup-completion state;
14. Setup completion alone does not trigger AP shutdown; an active AP is stopped when internet connectivity returns;
15. A configured device can start the Setup AP when offline; incomplete setup with working internet does not independently trigger startup AP creation;
16. direct setup continuation when setup AP fails;
17. SFTP provisioning is tested for repeated execution and partial failure. Full idempotency remains a future requirement; the current implementation rejects an already-existing requested username and does not validate SSH configuration with sshd -t before restart.
18. remote PathFinder registration rejection;
19. Older encrypted configuration schemas migrate to schema 2 and protected client key files; raw plaintext configuration is rejected by the current loader;
20. update artifact mismatch rejection.
21. Setup AP startup using ap0 interface.
22. Setup AP fallback when wlan0 is unavailable.
23. ap0 interface creation from wlan0.
24. Setup AP recovery after rfkill-blocked Wi-Fi.
25. Setup AP recovery after NetworkManager restart.
26. Setup AP cleanup after internet connectivity returns.
27. Engine startup continues when Setup AP recovery fails.
28. PathFinder reconnects using fixed 20 second delay with jitter.
29. Engine continues local operation when PathFinder provider initialisation fails.
30. Failed PathFinder provider does not remain published in ClientManager.
31. WebSocket close frame triggers clean disconnection handling.
32. Malformed WebSocket JSON packet does not kill the listener.
33. Heartbeat sender does not consume a CPU core while PathFinder is disconnected.
34. UpdateService does not consume a CPU core during idle polling.
35. CONNECT_TO_WIFI creates a Roof-owned NetworkManager profile.
36. CONNECT_TO_WIFI deletes stale Roof-owned profile before reconnecting.
37. CONNECT_TO_WIFI successfully brings the created profile up on wlan0.
38. CONNECT_TO_WIFI handles harmless nmcli rescan output without failing.
39. macOS local discovery completes using ARP candidates without subnet ping sweep.
40. local FORGET_CURRENT_WIFI deletes the connection profile active on wlan0;

41. remote FORGET_CURRENT_WIFI is rejected;
42. Successful RESTART_ROOF sends its response before requesting coordinated shutdown, and the operating-system reboot command is attempted after the shutdown sequence;
43. remote RESTART_ROOF behavior matches the approved authorisation policy;
44. Setup AP monitoring responds to internet loss and restoration across multiple timer cycles;
45. rotated logs are compressed into .log.zst files;
46. compressed logs older than the retention period are deleted;
47. ResourceManagerService starts as a registered service;
48. configuration schema 1 client credentials migrate into schema 2 client key files.

Current loader rejects raw plaintext configuration. Plaintext-to-encrypted migration tests remain pending until that feature is implemented.

21.4 Runtime Validation

Runtime validation should cover:

- Raspberry Pi 5 boot;
- startup service activation;
- OpenSSH/SFTP readiness;
- RoofDeck local discovery;
- local client registration;
- file create/update/delete;
- restart after update;
- network loss and recovery;
- PathFinder disconnection and reconnection;
- Wi-Fi listing and local connection;
- log generation;
- recovery after invalid command packets.

21.5 Acceptance Criteria

Minimum acceptance criteria should include:

- Engine starts on target Linux hardware;
- config is created if missing;
- local presence discovery works;
- managed TCP command channel accepts clients;
- first-time registration completes;
- registered client can connect;
- encrypted command packets are processed;
- the shared playground is provisioned with the expected group permissions, ACL inheritance, and access for registered client accounts;
- file CRUD commands work;
- PathFinder connection works when a valid, reachable WebSocket endpoint has been supplied in source and the required network path is available;
- local operation continues without PathFinder;

- update hash mismatch does not replace binary;
- malformed packets do not crash Engine;
- logs are written to file;sensitive configuration values are encrypted at rest;
- configuration integrity validation blocks tampered config startup;
- Engine does not continue with empty config after critical config failure;
- remote PathFinder-routed registration is rejected;
- In IA, Beta and Release builds, offline startup attempts Setup AP creation and activation regardless of setup-completion state;
- Runtime monitoring attempts to start an inactive Setup AP when internet connectivity is unavailable;
- Setup AP uses the dedicated ap0 interface;
- AP activation checks wireless readiness, ensures ap0 exists and validates or recreates the RoofAP profile. Failed creation or activation may trigger guarded wireless recovery;
- Failed disruptive recovery suppresses further recovery attempts until successful activation or already-active AP handling resets the recovery flag;
- Setup AP cleanup stops the AP after internet connectivity is restored;
- The RoofAP NetworkManager profile has autoconnect disabled. Engine may explicitly activate it during offline startup or runtime monitoring;
- Engine startup continues safely when Setup AP creation, startup, or recovery fails.invalid update artifacts are deleted and not applied;
- SFTP user setup is idempotent;
- NativeAOT build succeeds on .NET 10.
- PathFinder WebSocket listener port uses 8071;
- PathFinder startup logs use the shared WebSocket TCP listener port constant;
- failed PathFinder provider initialisation is cleaned up before retry;
- ClientManager only receives a successfully initialised PathFinder provider;
- PathFinder reconnect delay is fixed at 20 seconds with up to 20 percent jitter;
- heartbeat loops do not spin continuously while disconnected or idle;
- UpdateService uses PeriodicTimer and supports cancellation;
- WebSocket close frames, non-open states, and WebSocketException cases trigger disconnection handling;
- malformed WebSocket JSON does not terminate the listener;
- CONNECT_TO_WIFI validates arguments before indexing;
- CONNECT_TO_WIFI creates and activates a Roof-owned Roof-WiFi-{ssid} NetworkManager profile;
- Wi-Fi rescan does not fail on harmless non-empty nmcli output;
- macOS local-device discovery does not perform active subnet ping sweeps.
- the active configuration uses schema version 2;
- per-client AES network credentials are stored outside RegisteredClients in protected client key files;
- local registered clients can forget the Wi-Fi profile active on wlan0;
- PathFinder-routed clients cannot execute FORGET_CURRENT_WIFI;
- RESTART_ROOF synchronises OS state, waits five seconds and sends its response before requesting coordinated shutdown. Reboot is attempted after the shutdown sequence;

- the approved local/remote authorisation policy for RESTART_ROOF is enforced;
- ClientManagerService evaluates Setup AP requirements every 15 seconds;
- Setup AP is stopped after internet connectivity returns;
- Setup AP startup is attempted when connectivity is unavailable;
- ResourceManagerService is registered and starts successfully;
- active logs rotate after 24 hours;
- rotated logs are compressed with zstd;
- compressed logs are retained for no more than 30 days under normal service operation.

22. Risks & Limitations

22.1 Technical Risks

Current technical risks:

- no formal test suite;
- static shared client state;
- background threads and async callbacks may hide exceptions;
- async void patterns;
- shared client collections have been made thread-safe, but concurrent client state mutation should continue to be stress-tested;
- recursive file indexing may become expensive;
- update polling may be too frequent;
- coordinated shutdown is implemented, but timeouts, active-work disposal ordering, and forced termination require validation;
- PathFinder reconnection behaviour has been simplified to fixed delay plus jitter, but should be observed under unstable internet conditions;
- failed provider cleanup must remain reliable to avoid stale PathFinder references;
- CPU-hogging loop regressions remain a risk in future heartbeat, polling, or background-service changes;
- macOS local discovery now depends on ARP table availability rather than active subnet probing.

22.2 Security Risks

Current security risks:

- first-time key exchange is still local pre-encryption and may require stronger pairing later;
- registered-client nonce challenge remains disabled until further decision;
- automatic encryption rotation is not enforced;
- Setup AP is open during setup or recovery access and must be strictly lifecycle-constrained;
- Setup AP may be started when internet is unavailable, so recovery-mode AP exposure must be carefully controlled;
- remaining shell-facing operations must continue to be treated as high-risk;
- remote sender trust depends on registered identifier and payload decryption;

- credential and config encryption behaviour should continue to be tested under migration and failure cases.
- Wi-Fi SSID and password handling still crosses privileged shell-command boundaries;
- shell quoting has been added for Wi-Fi profile creation, but Wi-Fi command construction should continue to be reviewed;
- Roof-created Wi-Fi profile names include the SSID and should not expose passwords or secrets in logs.
- RESTART_ROOF currently has no remote-mode restriction, allowing any successfully authenticated registered client, including a PathFinder-routed client, to request an operating-system reboot;
- reboot authorisation is not separated from ordinary registered-client command authorisation;
- FORGET_CURRENT_WIFI and RESTART_ROOF cross privileged operating-system command boundaries;
- client encryption key files depend on filesystem ownership and mode 600 for at-rest isolation;
- static IV storage and reuse behavior should be reviewed as part of the wider cryptographic protocol assessment.

22.3 Reliability Risks

Current reliability risks:

- PathFinder dependency for remote access;
- local network discovery may fail in unusual network configurations;
- Setup AP creation may fail depending on wireless hardware, wlan0 availability, ap0 creation support, rfkill state, or NetworkManager state;
- heuristic Setup AP cleanup depends on accurate internet availability detection;
- update replacement may fail mid-flow after artifact verification;
- file cache can diverge from physical filesystem;
- large file transfer is not chunked;
- external filesystem changes are not continuously reconciled.
- Wi-Fi connection success depends on NetworkManager radio state, wlan0 managed state, stale profile cleanup, and wireless stack settle timing;
- PathFinder availability depends on correct detection of active WebSocket state after initialisation;
- macOS discovery may miss devices that are not present in the ARP table.
- RESTART_ROOF reports success before the reboot command's result is known;
- log rotation depends on ResourceManagerService continuing to execute;
- log compression depends on zstd being installed and callable;
- uncompressed logs left after compression failure are not covered by the current *.log.zst retention scan;
- log rotation and retention state is held in process memory and resets when Engine restarts.

22.4 Operational Risks

Current operational risks:

- Linux-only assumptions;
- Raspberry Pi resource constraints;
- Setup AP depends on compatible wireless hardware, NetworkManager AP support, rfkill, iw, ip, wlan0 availability, and successful ap0 interface creation;
- manual recovery paths;
- limited observability;
- limited health reporting;
- systemd detached process supervision concern;
- dependency on nmcli, OpenSSH, NetworkManager, and Linux utilities.
- zstd is now a runtime dependency;
- operators must account for both .log and .log.zst files during diagnostics and recovery;
- the log directory requires sufficient space for an active log and at least one uncompressed rotation candidate;
- ResourceManagerService currently runs at highest thread priority despite performing periodic file-management operations;
- remote reboot capability may interrupt active client operations if command authorisation is not further constrained.

22.5 Protocol Compatibility Risks

Current protocol risks:

- command prefixes are string-based;
- response payloads are positional;
- packet versioning is not formally defined;
- file metadata contracts may evolve without compatibility controls.

Appendices

Appendix A: Important Constants

Constant	Value
Local presence UDP port	6969
Local managed TCP port	8671
PathFinder URL	PATHFINDER_PUBLIC_URL - source placeholder requiring replacement with a valid WebSocket endpoint.
PathFinder identifier	43627a1de2ca425e9bf877b80dc45725
Config schema version	2
Startup unit path	/etc/systemd/system/roofenginestartup.service
Managed TCP data prefix	MTCP_DATA:
Managed TCP heartbeat	MTCP_HEARTBEAT
Managed TCP max frame	64 MiB
Managed UDP data prefix	MUDP_DATA:
Managed UDP ACK prefix	MUDP_ACK:
Managed UDP heartbeat	MUDP_HEARTBEAT
Managed UDP retransmission interval	2 seconds
Managed UDP max retries	5
WebSocket heartbeat	WS_HEARTBEAT

Constant	Value
WebSocket disconnection monitor polling	1000ms
PathFinder client heartbeat	PF_HEARTBEAT
Default heartbeat recurrence	5 seconds
Default disconnection timeout	15 seconds
File hashing buffer	128 KiB
Update tick	60 seconds
Setup AP internal connection name	RoofAP
Setup AP SSID	Roof Setup
Setup AP mode	802-11-wireless.mode ap
Setup AP autoconnect	disabled
Setup AP security	open network
WLAN recovery	Failed disruptive recovery is suppressed until successful or already-active AP handling resets the recovery flag.
Ap0 recovery	Interface existence and activation are checked during AP activation; creation has no permanent once-per-runtime restriction.
Setup AP requirement monitor	15 seconds
Wi-Fi connection profile prefix	Roof-WiFi-
Wi-Fi radio settle delay before connection	1500ms

Constant	Value
macOS local discovery source	ARP table candidates
Configuration encryption key	/etc/Roof/keys/roof_config.key
Client network encryption key	/etc/Roof/keys/{clientId}.key
Key-file binary layout	32-byte AES-256 key followed by 16-byte IV
Key-file permission mode	600
Resource Manager tick	60 seconds
Log rotation interval	24 hours
Compressed-log retention	30 days
Compressed-log filename pattern	*.log.zst
Log compression command	zstd -3 -q --rm
Wi-Fi forget command	FORGET_CURRENT_WIFI
Roof restart command	RESTART_ROOF
Restart pre-reboot delay	5 seconds
Remote client payload text limit	16 MiB of UTF-8 data
Encryption key directory permission mode	700

Appendix B: Glossary

Term	Meaning
Roof Engine	Device-side runtime process for Roof.
Roof Client	Registered application/client communicating with Engine.
RoofIdentifier	Engine/device identifier stored in config.
Client ID	32-character client identifier.
PathFinder	Remote WebSocket relay used for non-local communication.
RNMS	Roof Network Messaging System / NetworkMessageProvider abstraction.
Presence Beam	UDP discovery probe used to locate a local Roof device.
Managed TCP	Length-prefixed TCP transport with heartbeat and timeout.
Managed UDP	UDP transport with ACK/retry reliability.
RoofWebSocketPacket	PathFinder relay packet envelope.
Playground	Shared writable storage at /srv/roof/playground. Registered client accounts retain separate identities and credentials but access the same stored files.
Hashed Path	xxHash64 identifier generated from a file or directory path.
Native Packet	Roof JSON command or response packet.
NativeAOT	.NET ahead-of-time publishing model.
Setup AP	Temporary open wireless access point used for local onboarding and recovery. Current lifecycle decisions follow internet availability. Startup creation is attempted in IA, Beta and Release builds; periodic runtime monitoring also operates in Debug.

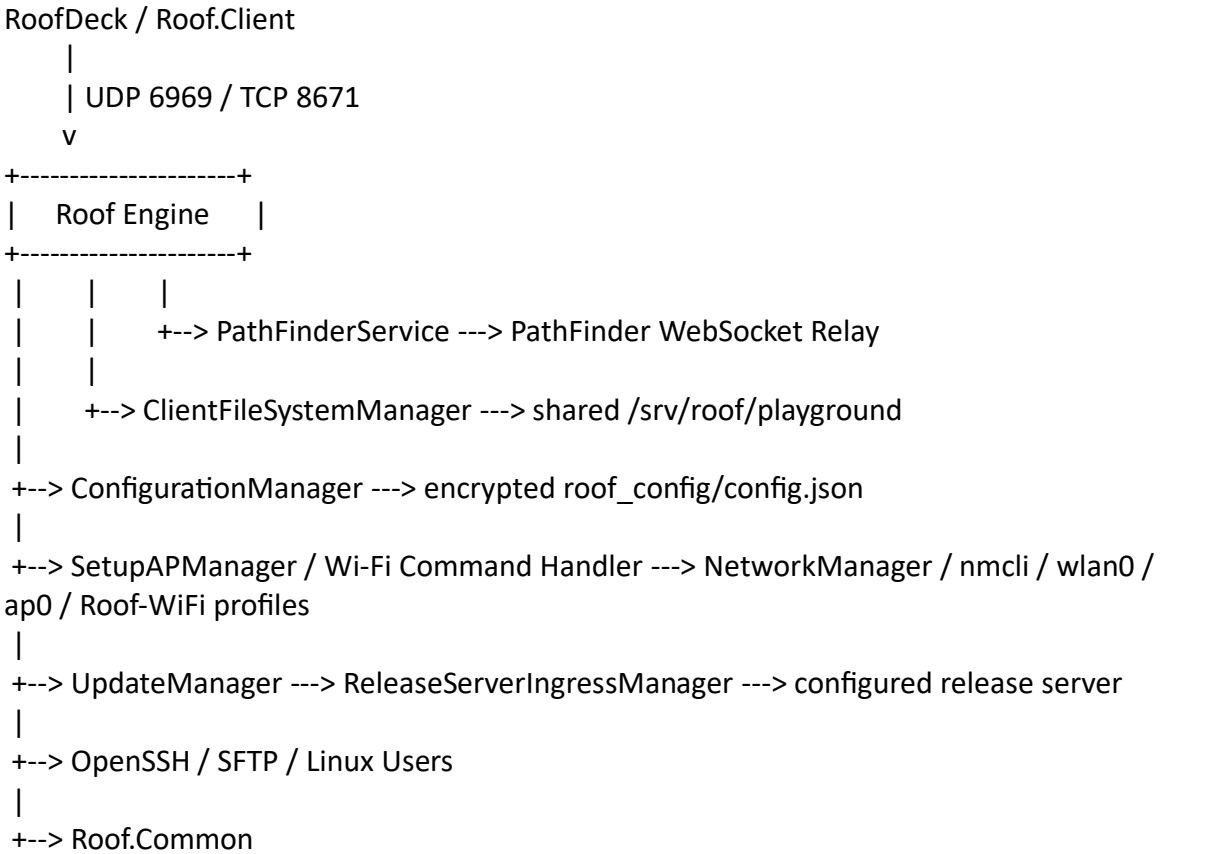
Term	Meaning
Setup Mode	The Engine state where HasCompletedInitialSetup is false and initial onboarding is not yet complete.
Ap0	Dedicated AP-mode wireless interface created from wlan0 where supported. Used by Roof Engine for Setup AP operation.
WLAN recovery	Guarded recovery that can unblock Wi-Fi, conditionally restart NetworkManager, enable the radio, bring wlan0 online, recreate ap0 and recreate the AP profile. Failed recovery suppresses further recovery attempts until successful or already-active AP handling resets the flag.
Setup AP Heuristics	Runtime logic that evaluates internet availability, AP state, wireless readiness, profile validity and recovery state. Setup completion is not currently an independent AP lifecycle condition.
Configuration-at-Rest Encryption	Encryption of sensitive Engine configuration values before persistence to config.json.
Configuration Integrity Enforcement	Startup-time validation that persisted configuration has not been tampered with or corrupted.
Roof Wi-Fi Profile	A NetworkManager connection profile created and owned by Roof Engine for Wi-Fi connection management. Current profile naming pattern is Roof-WiFi-{ssid}.
ARP-table Discovery	The macOS local-device discovery approach that uses ARP table candidates instead of active ICMP subnet ping sweeps.
PeriodicTimer Update Loop	The current UpdateService polling model based on .NET PeriodicTimer and cancellation support, replacing the earlier manual Thread and Stopwatch loop.

Appendix C: Acronyms

Acronym	Meaning
AES	Advanced Encryption Standard
API	Application Programming Interface

Acronym	Meaning
DTO	Data Transfer Object
IV	Initialisation Vector
JSON	JavaScript Object Notation
OS	Operating System
RNMS	Roof Network Messaging System
SFTP	SSH File Transfer Protocol
SHA	Secure Hash Algorithm
TCP	Transmission Control Protocol
UDP	User Datagram Protocol
URI	Uniform Resource Identifier
WebSocket	Full-duplex communication protocol over TCP

Appendix D: Component Diagram



The current source contains placeholders for the PathFinder endpoint and release server base address. These must be replaced with valid deployment addresses before remote connectivity and updates can operate.

Appendix E: Boot Sequence Diagram

Start Engine
  |
  v
Log Version / Compile Flag
  |
  v
Security Check
  |
  v
Device Check
  |
  v
Internet Check
  |
  v
Release Update Check, if applicable
  |
  v
OpenSSH / SFTP Setup
  |
  v
Systemd Startup Service Setup
  |
  v
Configuration Initialisation
  |
  v
Encrypted Config Load + Integrity Validation
  |
  v
IA/Beta/Release and internet unavailable: attempt Setup AP creation and activation
  |
  v
Migrate legacy playgrounds when applicable
  |
  v
Provision shared playground, permissions, ACLs and SFTP configuration
  |
  V
Restart SSH; abort service startup if provisioning or restart fails
  |
  V

Service Initialisation
|
v
Start Background Services
|
v
PathFinderService attempts provider initialisation
|
+–> success: publish provider after active connection
|
+–> failure: detach handlers, dispose provider, retry later
|
v
Wait for shutdown cancellation
|
v
Coordinated service termination, tracked-work wait and configuration save
|
v
Reboot if requested

Appendix F: Command Handling Diagram

Incoming Packet
|
+--> Local Managed TCP
|
+--> Remote PathFinder WebSocket
|
v
Validate Sender
|
v
Decrypt Payload with Registered Credentials
|
v
Deserialise Native Request Packet
|
v
Validate Command Prefix + Argument Count
|
v
Invoke Command Handler
|
v
Build Native Response Packet
|
v

```
Encrypt Response
        |
        v
Return via TCP or PathFinder
```

Appendix G: Local Registration Flow

```
Client              Engine
  |                   |
  | TCP connect :8671     |
  |---------------------->|
  |                   |
  | TCP INFO REQUEST      |
  |---------------------->|
  |                   |
  | TCP_ENCRYPTION_INFO   |
  |<----------------------|
  |                   |
  | Encrypted REGISTER    |
  | REQUEST             |
  |---------------------->|
  |                   |
  | Store client config   |
  |                   |
  | Encrypted success     |
  |<----------------------|
```

Appendix H: Remote PathFinder Flow

```
Remote Client
   |
   | RoofWebSocketPacket with encrypted payload
   v
PathFinder
   |
   | Relay packet
   v
Roof Engine PathFinderService
   |
   | Validate sender is registered
   v
ClientManager
   |
   | Decrypt and dispatch command
   v
Command Handler
   |
   | Encrypted response
   v
```

PathFinder
  |
  v
Remote Client